Portada

Universidad de El Salvador

Facultad Multidisciplinaria de Occidente

Departamento de Ingeniería y Arquitectura

Ingeniería en Desarrollo de Software / Educación En Línea

Introducción al Software Libre — Ciclo II 2026

Proyecto Final: Servidor Linux automatizado con Docker

Grupo: 7

Integrantes:

- Castillo Zepeda, Azucena del Carmen — CZ08008 — @cz08008-zuz
- Calzada Vargas, Carlos Enoc — CV19058 — @cecv9
- Cruz Arce, Josué Daniel — CA23002 — @DanielCruz98-503
- Portillo Consuegra, Ronnie Odir — PC15016 — @pc15016-Ronnie

Coordinadora de Cátedra: MEd. Angela López de Granillo

Tutor GT01: Francisco Javier Morales Ayala

Fecha de entrega: 03 de noviembre de 2026

Índice

Portada 1

Proyecto Final: Servidor Linux automatizado con Docker 1

Índice 2

Introducción 5

Objetivos 5

Objetivo general 5

Objetivos específicos 5

Marco teórico 5

Arquitectura de la solución 6

Fases del proyecto 6

Fase 00 — Preparación del entorno 6

Objetivo 6

Desarrollo 6

Comandos principales 6

Evidencias 7

Conclusión de la fase 10

Fase 01 — Administración básica, usuarios y permisos 10

Objetivo 10

Desarrollo 10

Comandos principales 10

Evidencias 11

Conclusión de la fase 12

Fase 02 — Automatización y monitoreo 12

Objetivo 12

Desarrollo 13

Comandos principales 13

Evidencias 13

Conclusión de la fase 15

Fase 03 — Control de versiones y GitHub 15

Objetivo 15

Desarrollo 15

Comandos principales 16

Evidencias 16
Conclusión de la fase 19
Fase 04 — Instalación y configuración de Docker 20
Objetivo 20
Desarrollo 20
Comandos principales 20
Evidencias 21
Conclusión de la fase 27
Fase 05 — Servidor web Nginx básico 27
Objetivo 27
Desarrollo 27
Comandos principales 27
Evidencias 27
Conclusión de la fase 29
Fase 06 — Imagen personalizada con Dockerfile 29
Objetivo 29
Desarrollo 29
Comandos principales 29
Evidencias 30
Conclusión de la fase 33
Fase 07 — Redes y volúmenes de Docker 33
Objetivo 33
Desarrollo 34
Comandos principales 34
Evidencias 35
Conclusión de la fase 42
Fase 08 — Pruebas y verificación final 42
Objetivo 42
Desarrollo 42
Comandos principales 42
Evidencias 43
Conclusión de la fase 49
Exportación e importación de la OVA 49

Objetivo 49
Desarrollo 50
Comandos principales 50
Evidencias 51
Conclusión de la fase 56
Resultados 56
Dificultades y soluciones 57
Conclusiones 57
Recomendaciones y trabajo futuro 58
Bibliografía 58
Anexos 58
Anexo A — Script de monitoreo 58
Anexo B — Dockerfile 59
Anexo C — Script de verificación integral 59
Anexo D — Repositorio 64

Introducción

El proyecto implementa un servidor Linux automatizado sobre Debian 13, dentro de una máquina virtual de VirtualBox. El servidor integra administración de usuarios y permisos, automatización con Bash y cron, control de versiones con Git y GitHub, y cuatro servicios web Nginx ejecutados con Docker.

La finalidad es demostrar, con evidencias verificables, que un servidor puede configurarse de forma reproducible, monitorearse sin intervención humana y recuperarse después de un reinicio. También se busca que cualquier integrante pueda importar la máquina virtual y obtener el mismo entorno de trabajo.

Objetivos

Objetivo general

Implementar un servidor Linux automatizado mediante Docker, integrando administración de usuarios y permisos, automatización con Bash y cron, control de versiones y servicios web containerizados.

Objetivos específicos

1. Configurar hostname, usuarios, grupos, sudo y permisos con herencia de grupo.
2. Automatizar un reporte del sistema cada 30 minutos.
3. Versionar el proyecto en Git y GitHub con commits de todos los integrantes.
4. Instalar Docker y desplegar cuatro servicios Nginx con técnicas distintas.
5. Construir una imagen personalizada, una red y un volumen persistente.
6. Verificar la conectividad y documentar cada fase con evidencias.

Marco teórico

- Debian 13: distribución de GNU/Linux estable, con apt, systemd, cron y herramientas de administración.
- VirtualBox: hipervisor que permite ejecutar la VM y tomar snapshots.
- Docker: plataforma de contenedores. Una imagen es la plantilla inmutable; un contenedor es una instancia en ejecución.
- Bind mount: montaje de una carpeta del host dentro del contenedor.
- Volumen: almacenamiento gestionado por Docker que persiste aunque el

contenedor se elimine.

- Red bridge personalizada: red de Docker con DNS interno para resolver

contenedores por nombre.

- Git y GitHub: control de versiones distribuido y repositorio remoto.
- cron: planificador de tareas del sistema.
- Nginx: servidor web ligero usado para publicar el sitio del proyecto.

Arquitectura de la solución

```
Host con VirtualBox
│  NAT: 2222→22, 8080→8080, 8081→8081, 8082→8082, 8083→8083
▼
VM Debian 13 «servidor-grupo7»
├── Docker Engine
│   ├── nginx-basico        :8080  bind mount /proyecto/web (ro)
│   ├── nginx-personalizado :8081  imagen servidor-grupo7 (COPY web/)
│   ├── nginx-red           :8082  red-proyecto
│   └── nginx-volumen       :8083  volumen-web
├── cron cada 30 min → /var/log/proyecto/reporte_sistema.log
└── Git /proyecto → GitHub cecv9/proyecto-linux-grupo7
```

Puerto	Contenedor	Técnica
8080	nginx-basico	Bind mount de /proyecto/web
8081	nginx-personalizado	Imagen propia servidor-grupo7
8082	nginx-red	Red personalizada red-proyecto
8083	nginx-volumen	Volumen persistente volumen-web

Fases del proyecto

Fase 00 — Preparación del entorno

Objetivo

Partir de una base limpia, actualizada y reproducible antes de configurar el servidor.

Desarrollo

Se actualizó Debian 13, se instalaron las herramientas base (curl, git, vim, wget, nano, cron y openssh-server), se configuró la zona horaria America/El_Salvador, se verificaron red, SSH y paquetes, y se tomó el snapshot 00-base-instalada.

Comandos principales

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git vim wget nano cron openssh-server
sudo timedatectl set-timezone America/El_Salvador
hostnamectl
ip a
systemctl status ssh --no-pager
sudo dpkg --audit
```

Evidencias

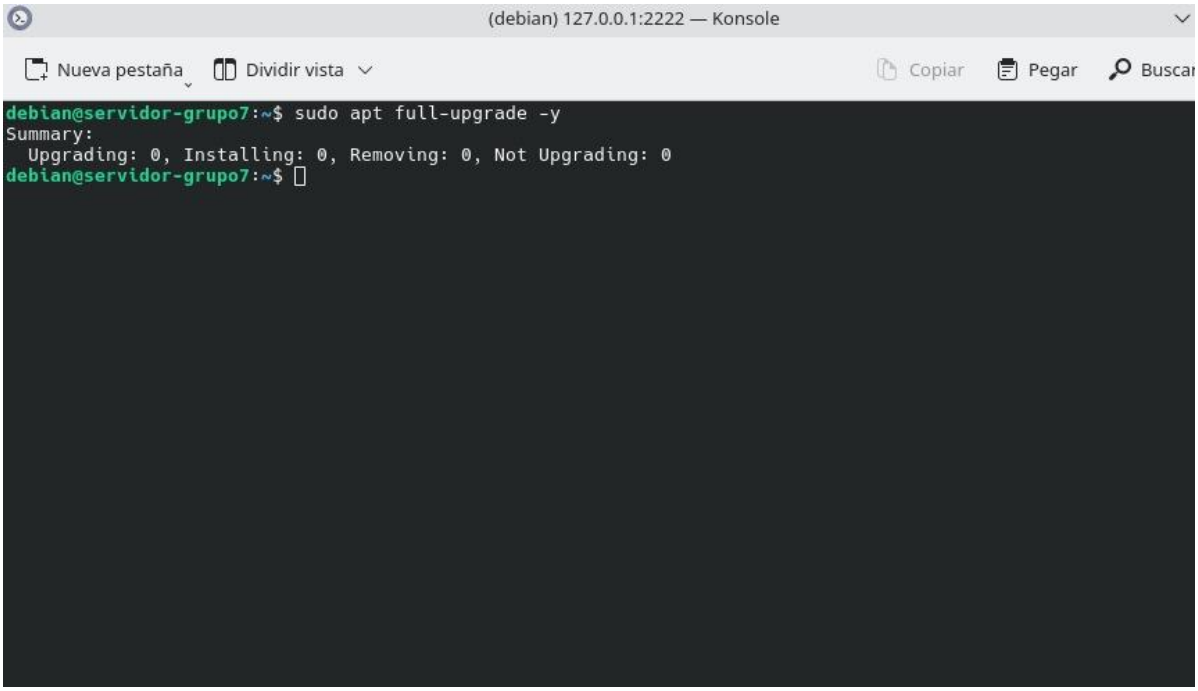


Figura 1. Actualización de paquetes del sistema con apt.

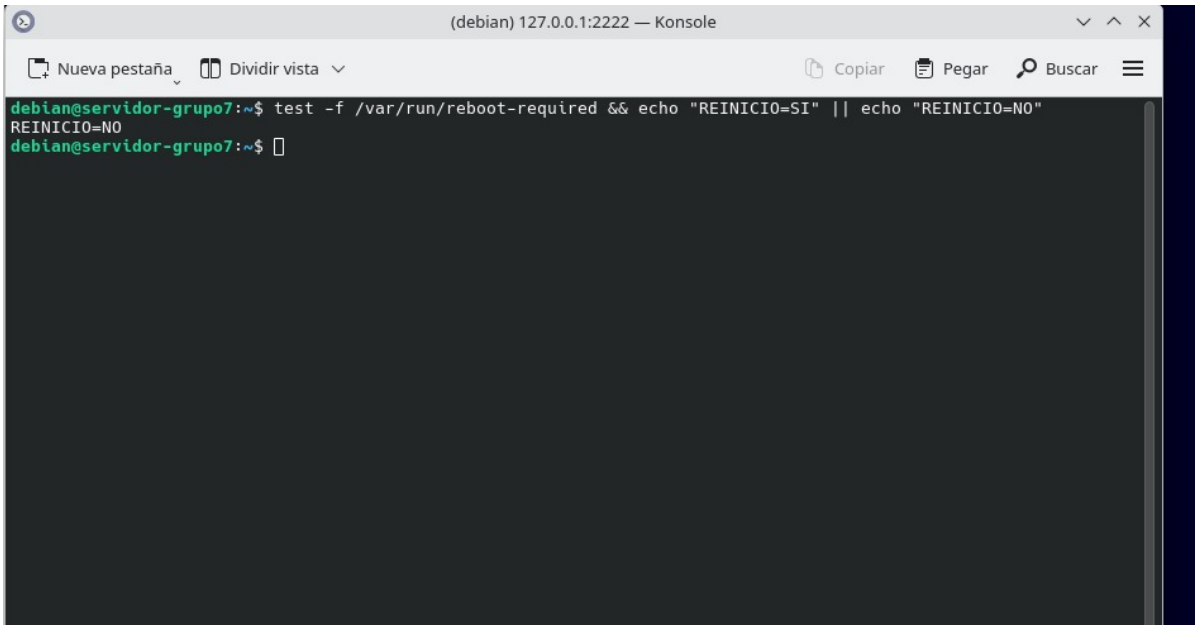


Figura 2. Reinicio de la máquina virtual después de actualizar.

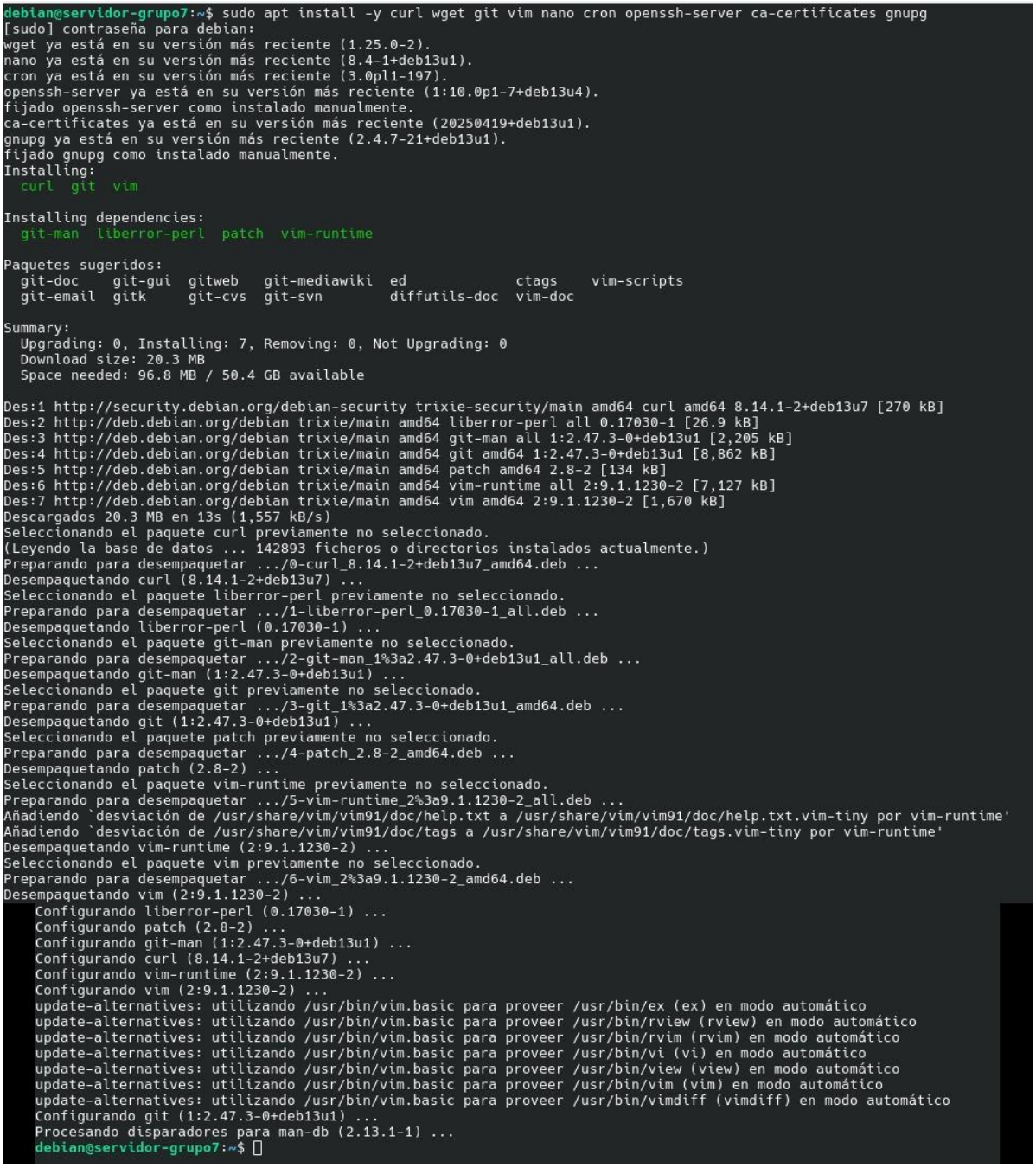


Figura 3. Instalación de curl, git, vim, wget, nano, cron y openssh-server.

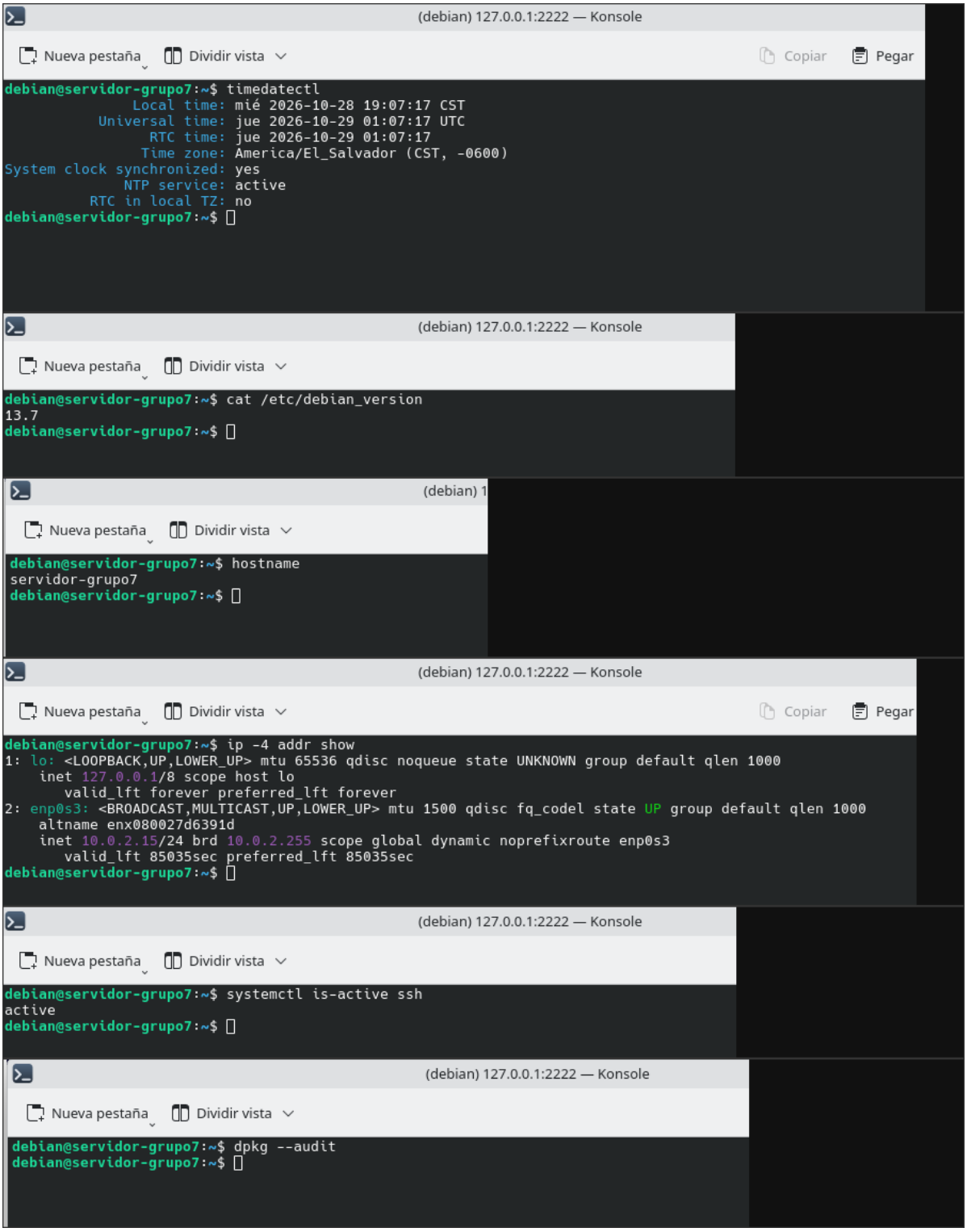


Figura 4. Resumen del entorno base: hostname, red, SSH y paquetes.

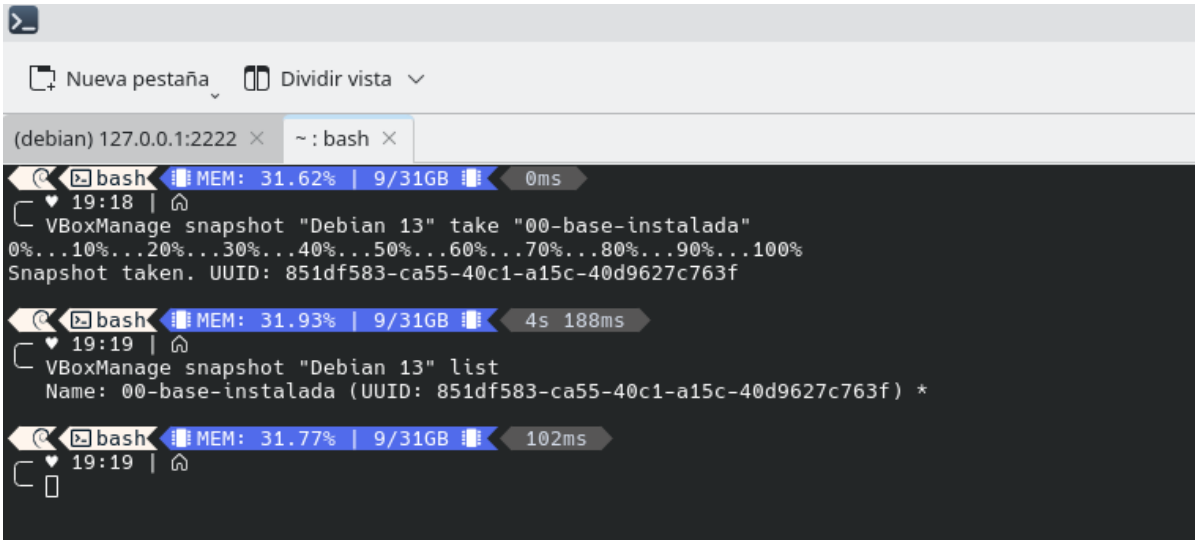


Figura 5. Snapshot 00-base-instalada tomado desde VirtualBox.

Conclusión de la fase

El entorno quedó listo y con un punto de restauración para revertir cambios si algo falla.

Fase 01 — Administración básica, usuarios y permisos

Objetivo

Dar identidad al servidor y separar los roles de administración, soporte y visitantes.

Desarrollo

Se configuró el hostname servidor-grupo7, se crearon los usuarios adminsys, tecnico y visitante, y los grupos soporte y web. adminsys quedó con sudo, tecnico en soporte y visitante en web. También se creó /proyecto/{datos,web,scripts,capturas}, se asignó datos al grupo soporte, web al grupo web y se activó el bit setgid (2775) para que los archivos nuevos hereden el grupo del directorio.

Comandos principales

```
sudo hostnamectl set-hostname servidor-grupo7
sudo adduser adminsys
sudo adduser tecnico
sudo adduser visitante
sudo addgroup soporte
sudo addgroup web
sudo usermod -aG sudo adminsys
sudo usermod -aG soporte tecnico
sudo usermod -aG web visitante
sudo mkdir -p /proyecto/{datos,web,scripts,capturas}
sudo chown adminsys:adminsys /proyecto
sudo chgrp soporte /proyecto/datos
sudo chgrp web /proyecto/web
sudo chmod 2775 /proyecto/datos /proyecto/web
```

Evidencias

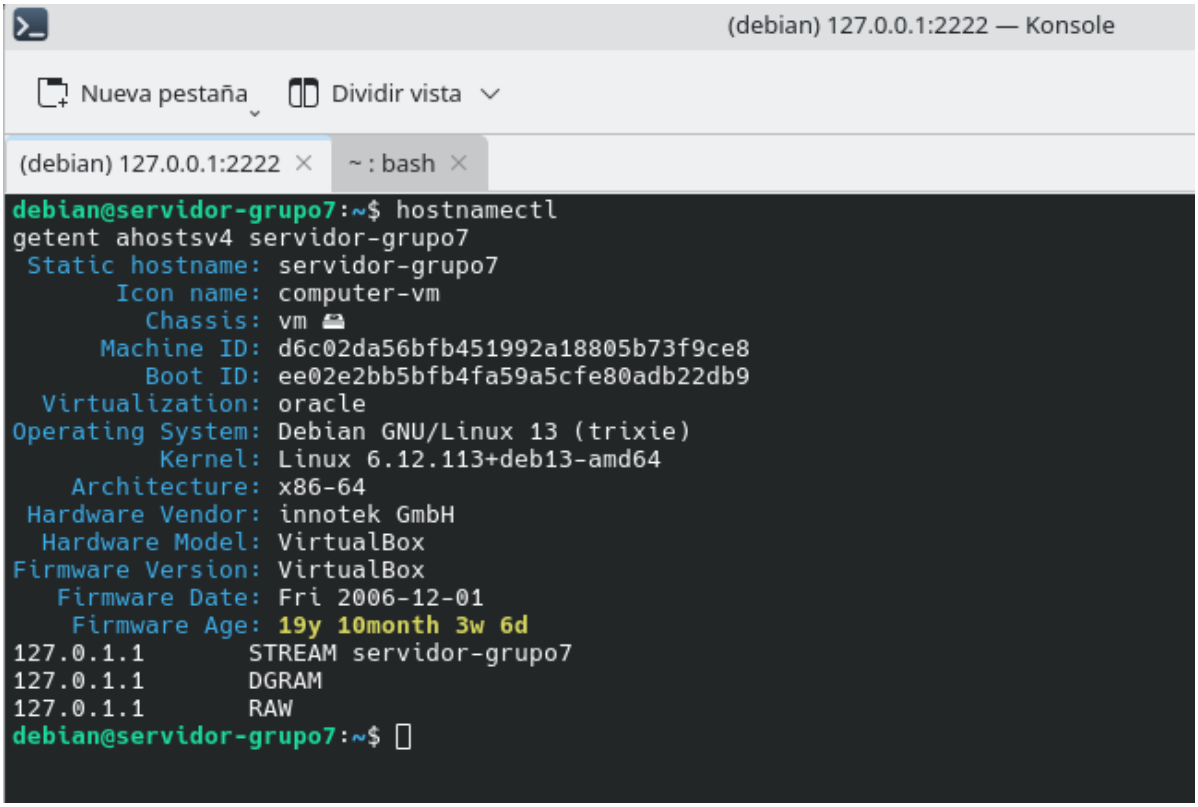


Figura 6. Hostname permanente servidor-grupo7 y resolución en /etc/hosts.

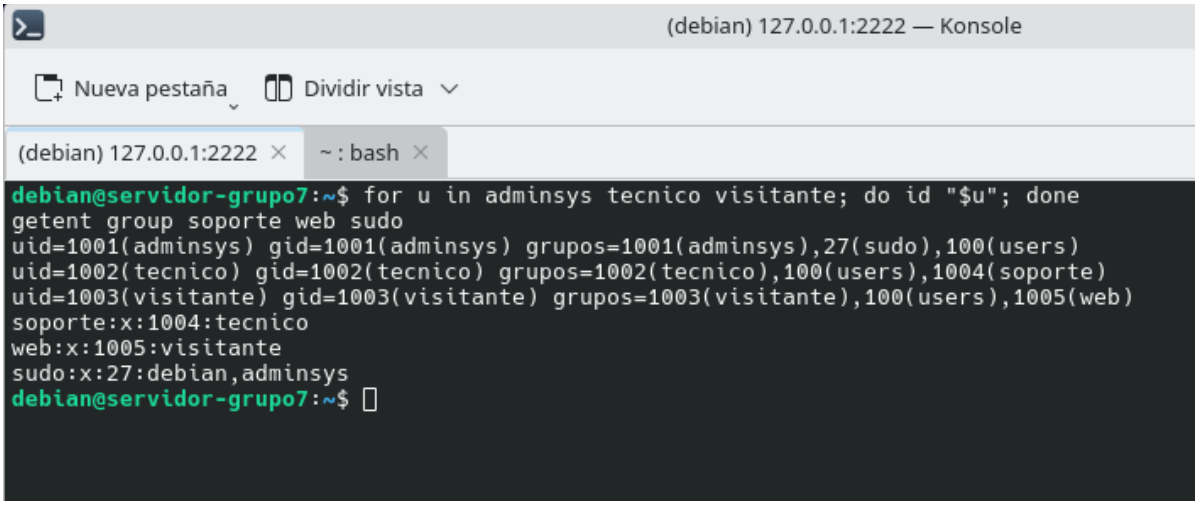


Figura 7. Creación de adminsys, tecnico, visitante y de los grupos soporte y web.

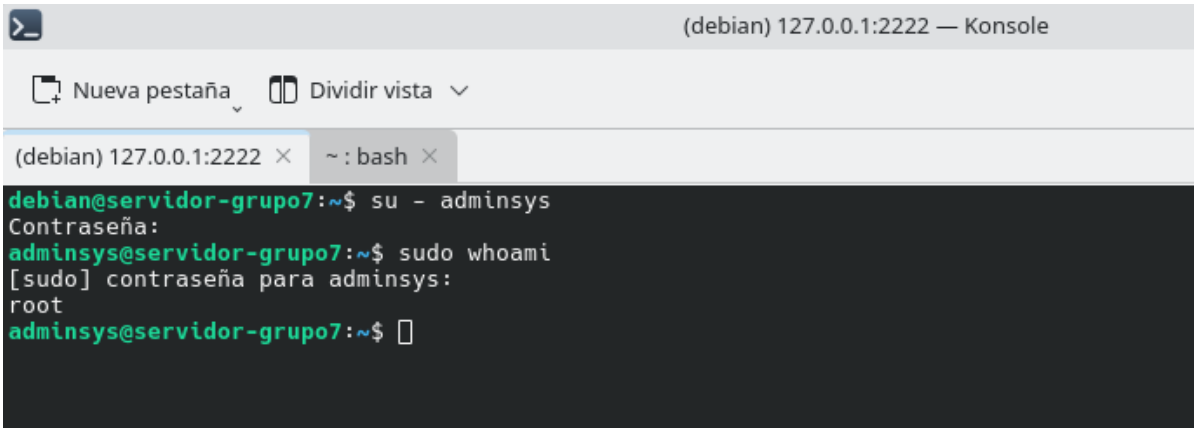


Figura 8. Comprobación de que adminsys pertenece al grupo sudo.

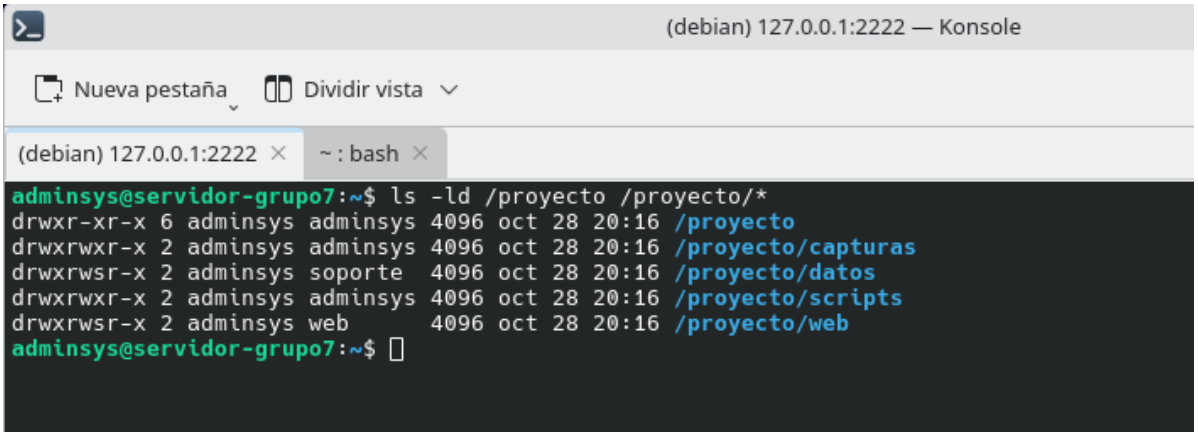


Figura 9. Estructura /proyecto y permisos de datos y web.

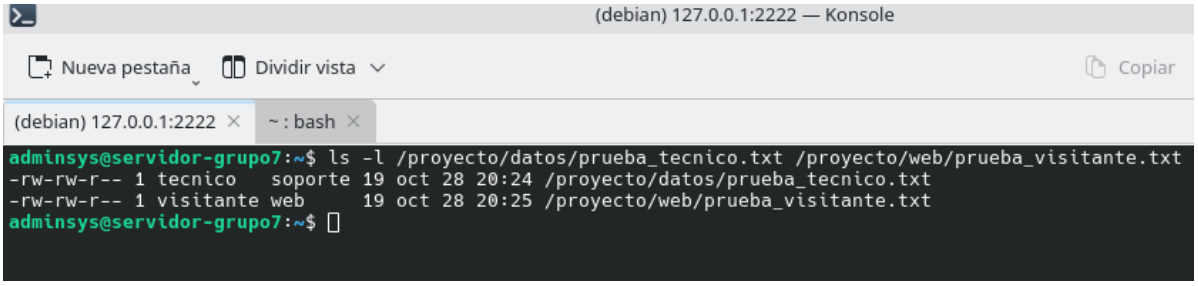


Figura 10. Prueba de herencia de grupo gracias al setgid 2775.

Conclusión de la fase

Cada rol tiene acceso a lo que le corresponde y la herencia de grupo queda garantizada por el setgid.

Fase 02 — Automatización y monitoreo

Objetivo

Registrar el estado del servidor sin intervención humana.

Desarrollo

Se creó reporte_sistema.sh, que muestra fecha y hora, hostname, usuarios conectados, espacio libre en disco, RAM disponible y cantidad de contenedores Docker activos. El script tiene permisos de ejecución y contempla el caso de que Docker no esté instalado o el daemon esté detenido. La tarea /etc/cron.d/proyecto lo ejecuta cada 30 minutos y agrega la salida a /var/log/proyecto/reporte_sistema.log.

Comandos principales

```
chmod 755 /proyecto/scripts/reporte_sistema.sh
bash -n /proyecto/scripts/reporte_sistema.sh
/proyecto/scripts/reporte_sistema.sh
sudo systemctl enable --now cron
sudo cat /etc/cron.d/proyecto
tail -n 30 /var/log/proyecto/reporte_sistema.log
```

Evidencias

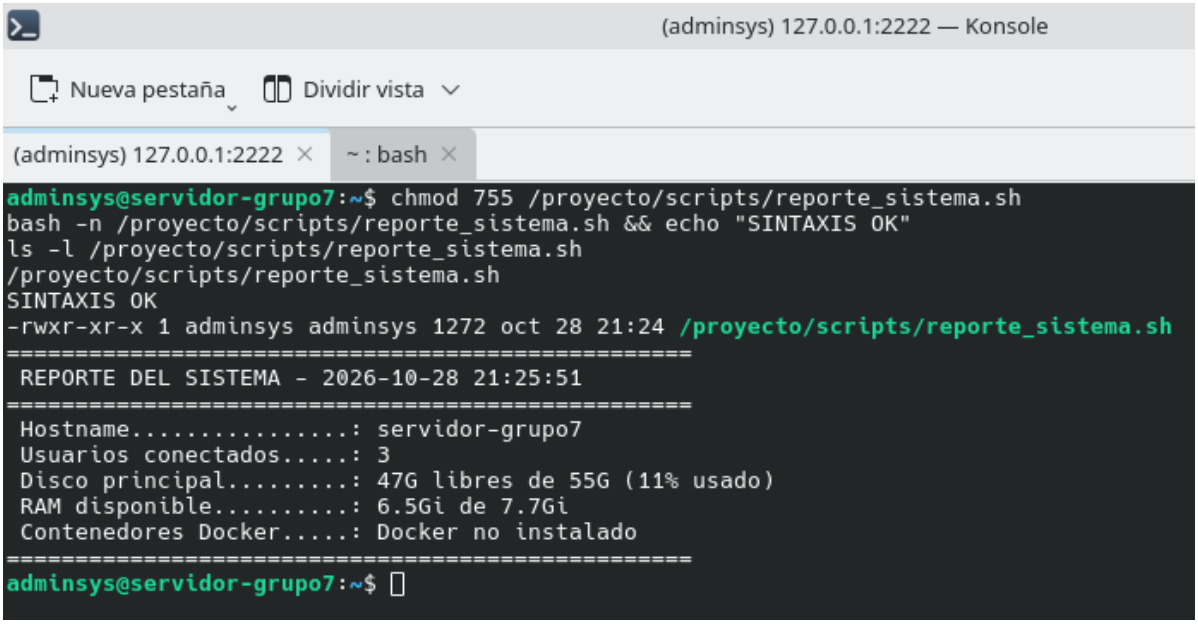


Figura 11. Script reporte_sistema.sh con permisos de ejecución.

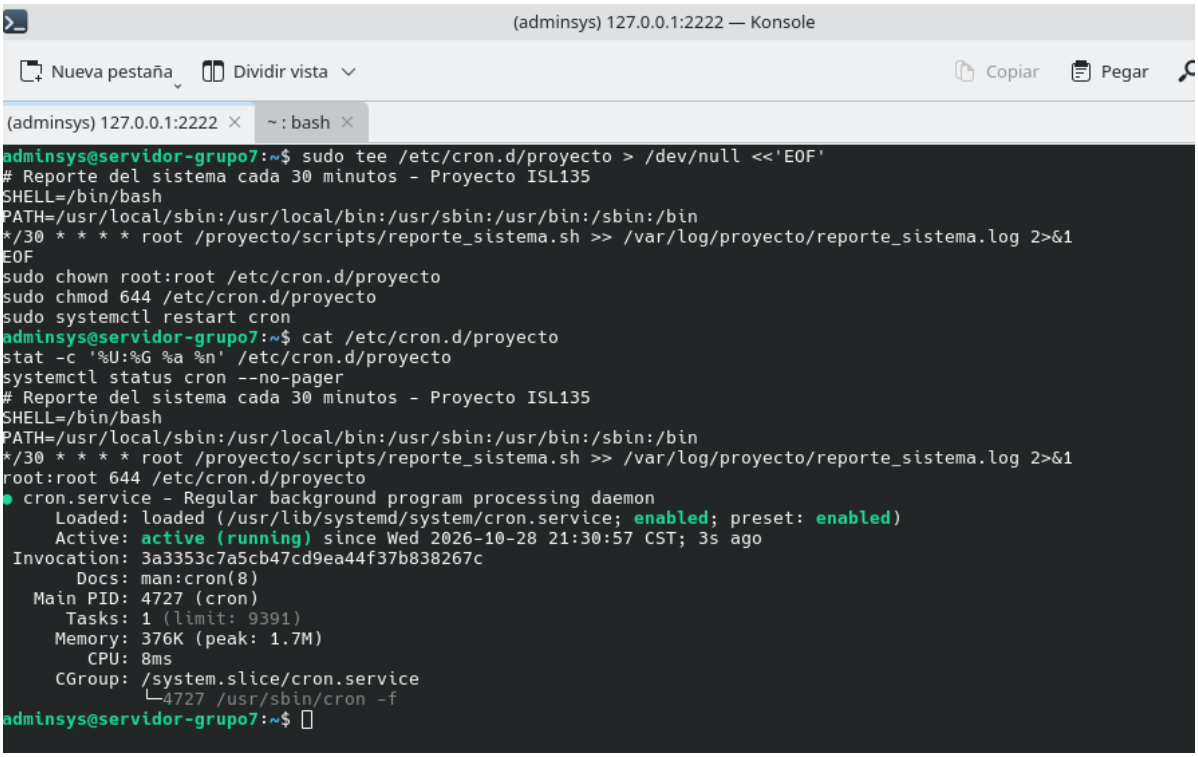


Figura 12. Tarea /etc/cron.d/proyecto programada cada 30 minutos.

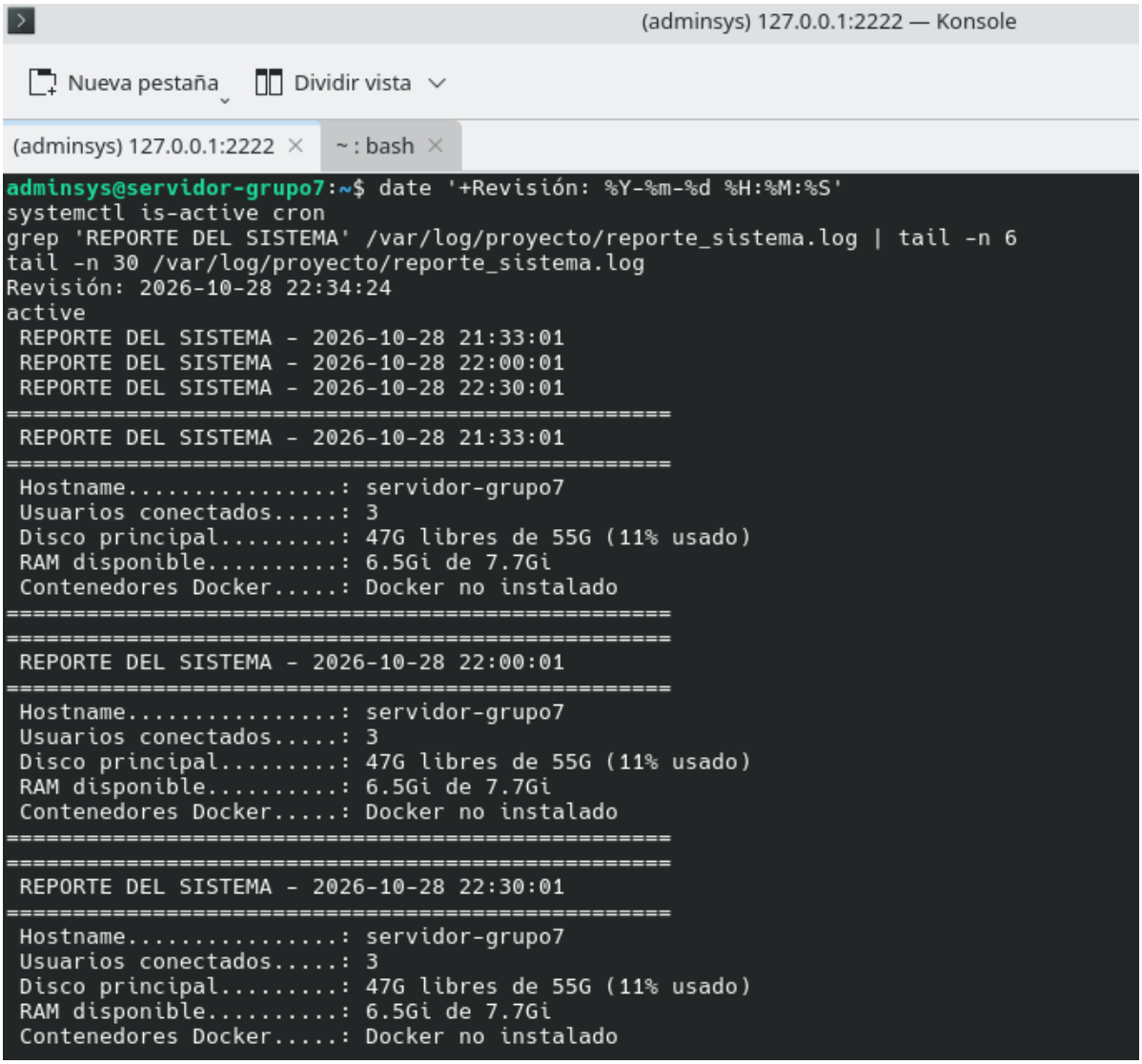


Figura 13. Log /var/log/proyecto/reporte_sistema.log actualizándose.

Conclusión de la fase

El estado del servidor queda registrado cada 30 minutos, aunque nadie esté conectado.

Fase 03 — Control de versiones y GitHub

Objetivo

Versionar todo el proyecto y demostrar trabajo colaborativo.

Desarrollo

Se inicializó el repositorio Git en /proyecto, se creó el primer commit con la estructura, se configuró el remoto en GitHub (cecv9/proyecto-linux-grupo7) y se autenticó por SSH. Se agregó el README.md con la descripción e instalación. El historial tiene 29 commits y participación de los 4 integrantes mediante ramas y pull requests.

Comandos principales

```
cd /proyecto
git init
git config user.name "Nombre del integrante"
git config user.email "correo@ues.edu.sv"
git add .
git commit -m "Estructura inicial del proyecto"
git remote add origin git@github.com:cecv9/proyecto-linux-grupo7.git
git push -u origin main
git log --oneline --graph
git shortlog -sne
```

Evidencias

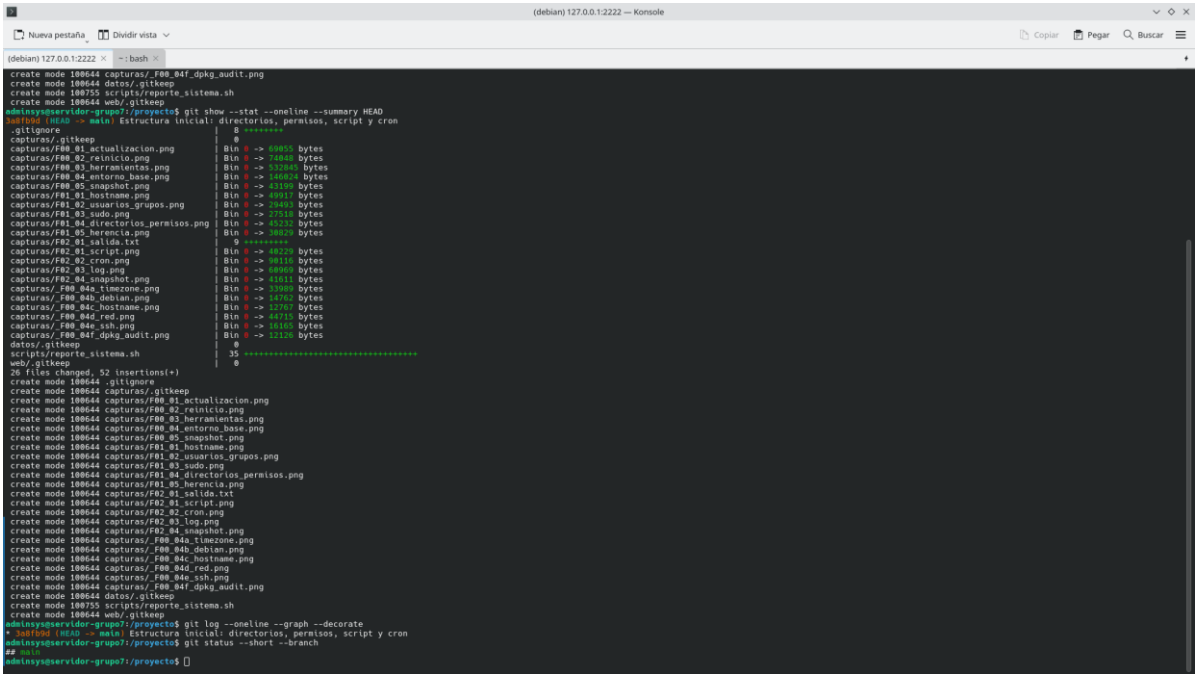


Figura 14. Repositorio Git local inicializado en /proyecto.

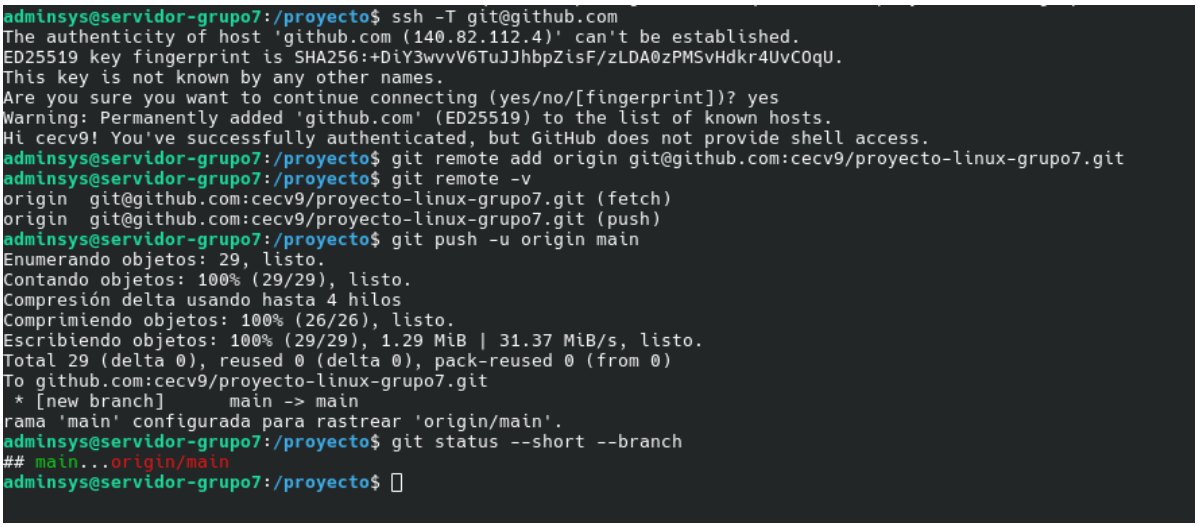


Figura 15. Repositorio remoto en GitHub y primer push.

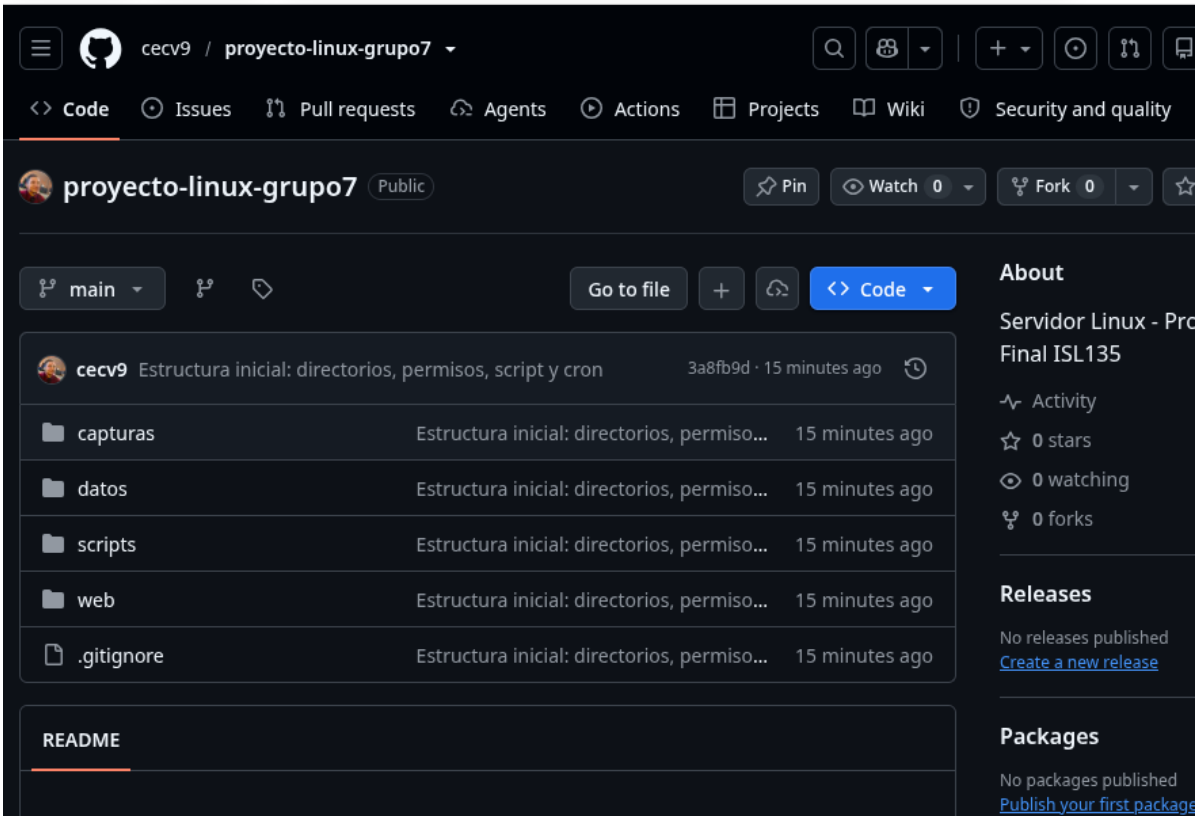


Figura 16. Vista del repositorio en GitHub.

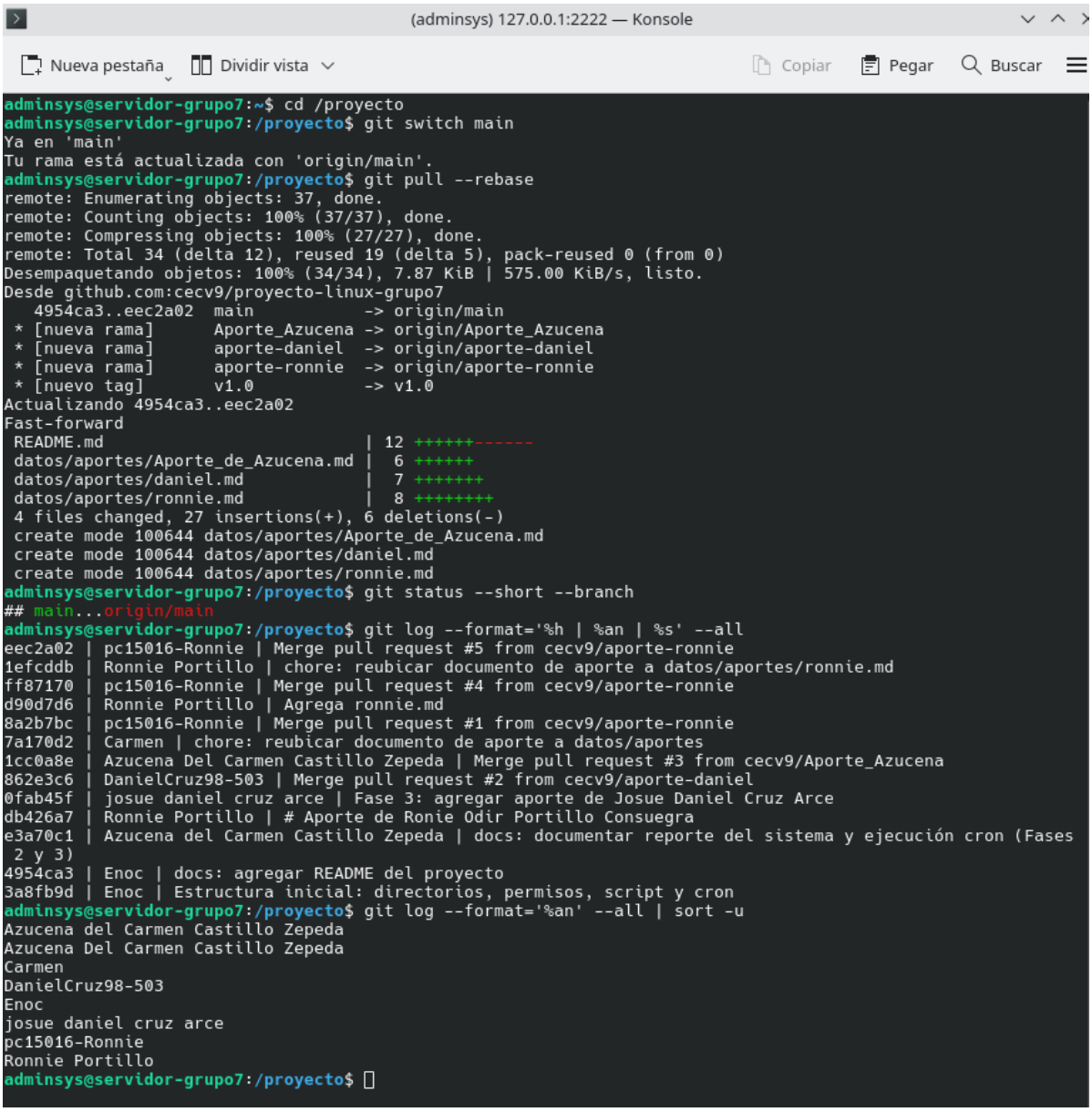


Figura 17. Historial con commits de los cuatro integrantes.

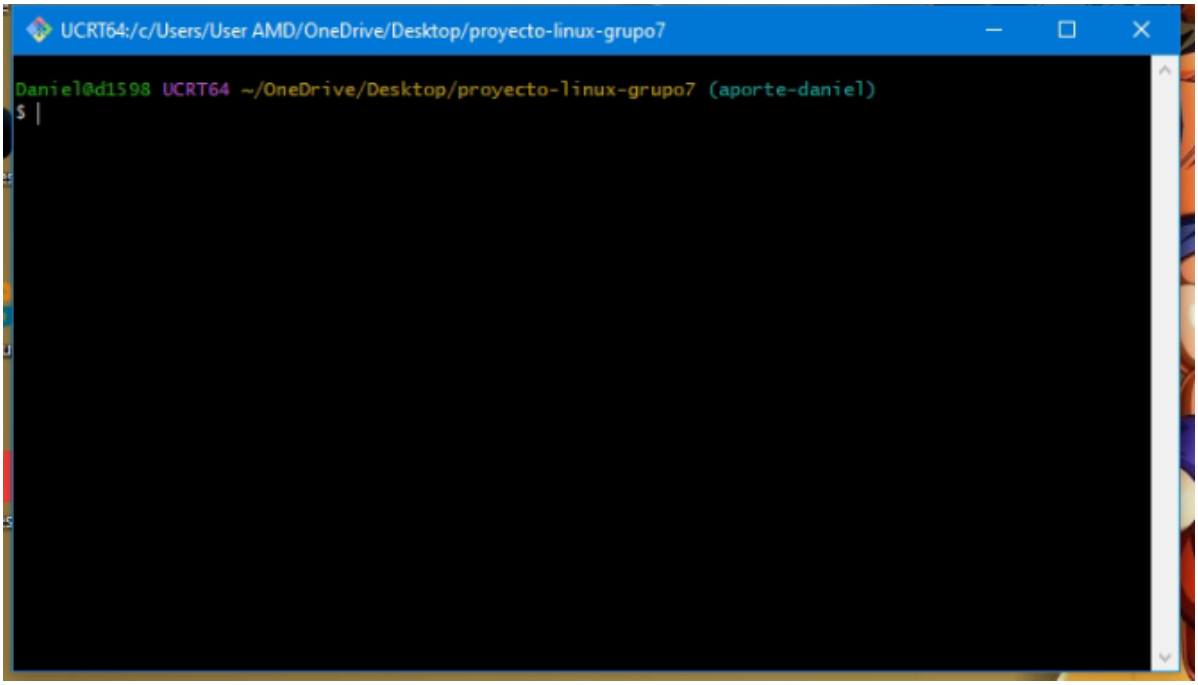


Figura 18. Rama de trabajo de Daniel.

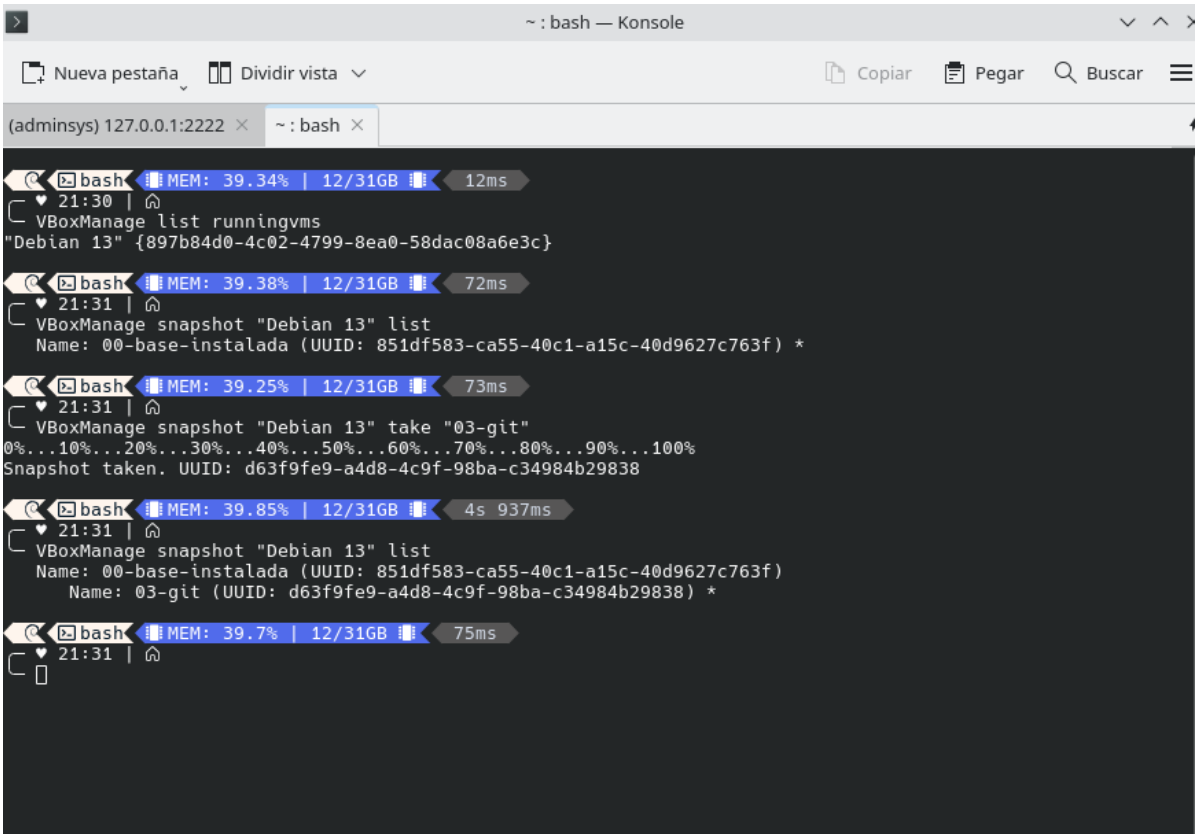


Figura 19. Snapshot de cierre de la fase 3.

Conclusión de la fase

El proyecto es trazable, reproducible y colaborativo desde GitHub.

Fase 04 — Instalación y configuración de Docker

Objetivo

Instalar Docker Engine, habilitarlo y permitir su uso sin sudo a los usuarios autorizados.

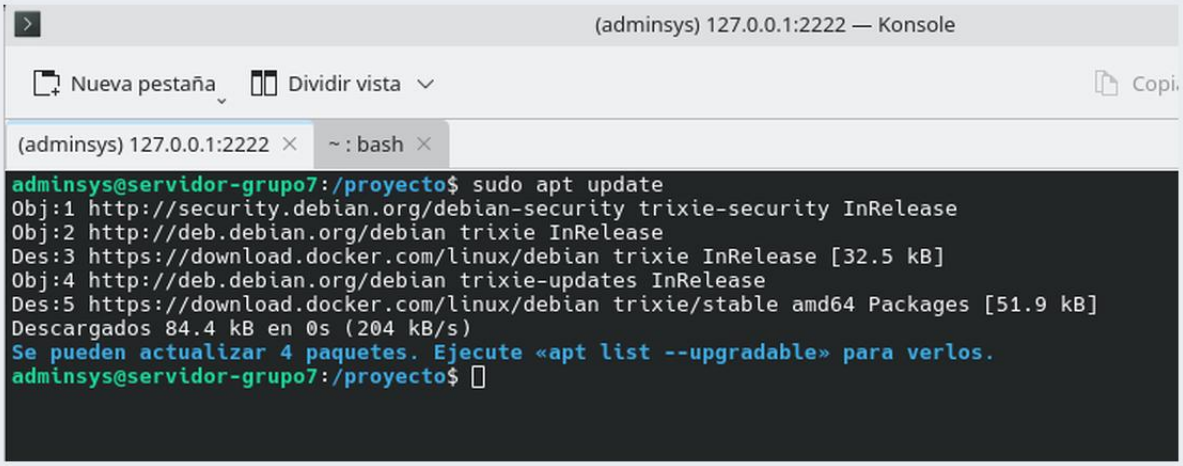
Desarrollo

Se instaló Docker desde el repositorio oficial, se habilitó e inició el servicio, se agregaron adminsys y tecnico al grupo docker, se aplicó el cambio con newgrp docker y se ejecutó la prueba oficial hello-world. Finalmente se listaron imágenes y contenedores.

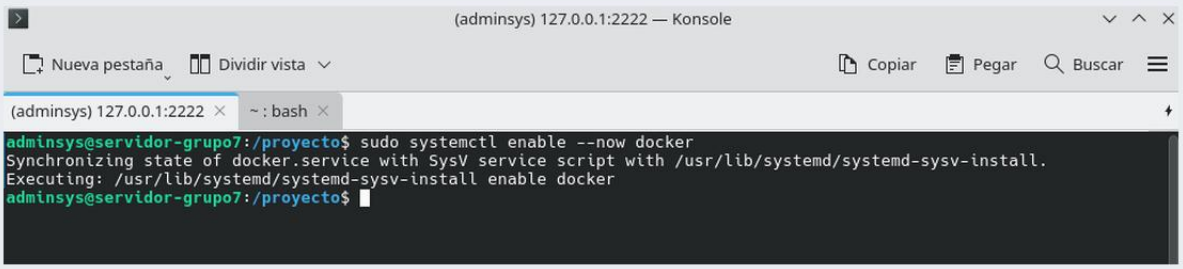
Comandos principales

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo usermod -aG docker adminsys
sudo usermod -aG docker tecnico
newgrp docker
docker run hello-world
docker ps -a
docker images
```

Evidencias



Repositorio oficial Docker leido por APT



Docker habilitado e iniciado

Figura 20. Salida de docker info con cliente y servidor.

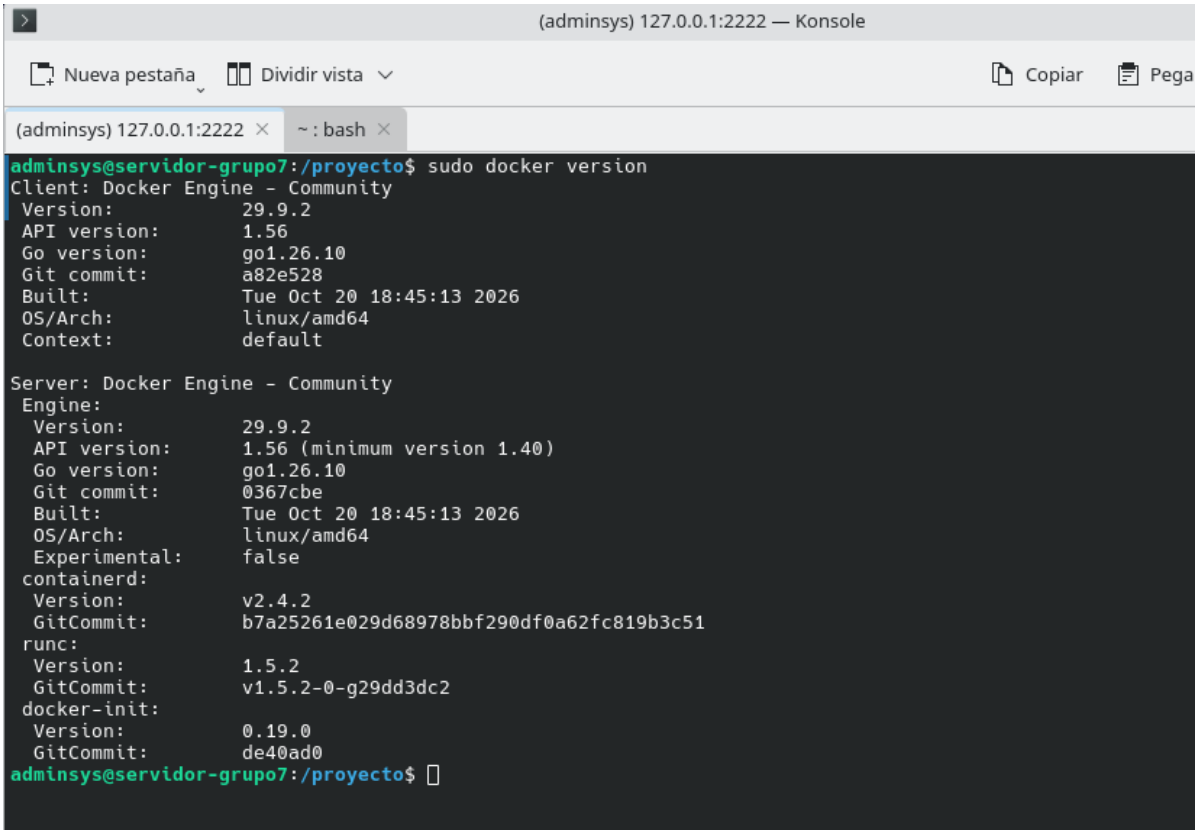


Figura 21. Versión de Docker.

(adminsys) 127.0.0.1:2222 — Konsole

Nueva pestaña Dividir vista

(adminsys) 127.0.0.1:2222 × ~ : bash ×

```
adminsys@servidor-grupo7:/proyecto$ sudo usermod -aG docker adminsys
adminsys@servidor-grupo7:/proyecto$ sudo usermod -aG docker tecnico
adminsys@servidor-grupo7:/proyecto$ getent group docker
docker:x:989:adminsys,tecnico
adminsys@servidor-grupo7:/proyecto$
adminsys@servidor-grupo7:/proyecto$
```

Grupo docker con adminsys y tecnico

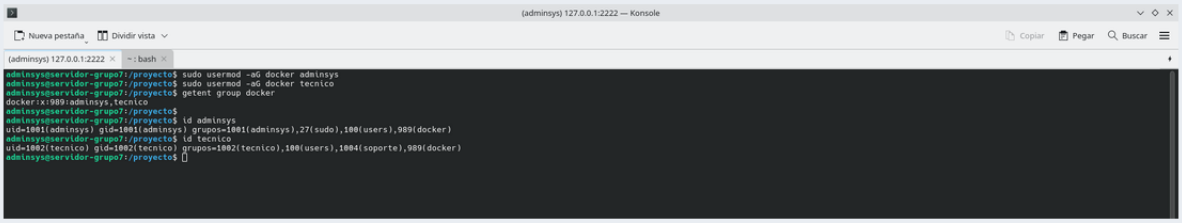
(adminsys) 127.0.0.1:2222 — Konsole

Nueva pestaña Dividir vista Copiar

(adminsys) 127.0.0.1:2222 × ~ : bash ×

```
adminsys@servidor-grupo7:/proyecto$ sudo usermod -aG docker adminsys
adminsys@servidor-grupo7:/proyecto$ sudo usermod -aG docker tecnico
adminsys@servidor-grupo7:/proyecto$ getent group docker
docker:x:989:adminsys,tecnico
adminsys@servidor-grupo7:/proyecto$
adminsys@servidor-grupo7:/proyecto$ id adminsys
uid=1001(adminsys) gid=1001(adminsys) grupos=1001(adminsys),27(sudo),100(users),989(docker)
adminsys@servidor-grupo7:/proyecto$
```

Pertenencia efectiva de adminsys



Pertenencia efectiva de tecnico

(adminsys) 127.0.0.1:2222 — Konsole

Nueva pestaña Dividir vista

(adminsys) 127.0.0.1:2222 × ~ : bash ×

```
adminsys@servidor-grupo7:/proyecto$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
adminsys@servidor-grupo7:/proyecto$
```

adminsys ejecuta docker ps sin sudo

(adminsys) 127.0.0.1:2222 — Konsole

Figura 22. Grupo docker.

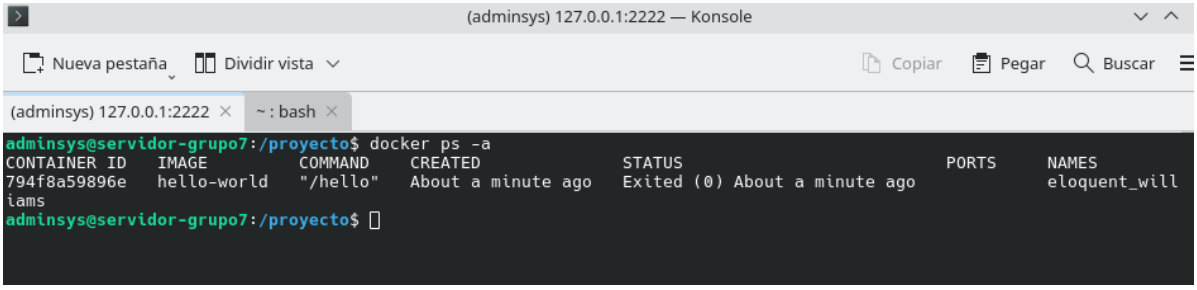
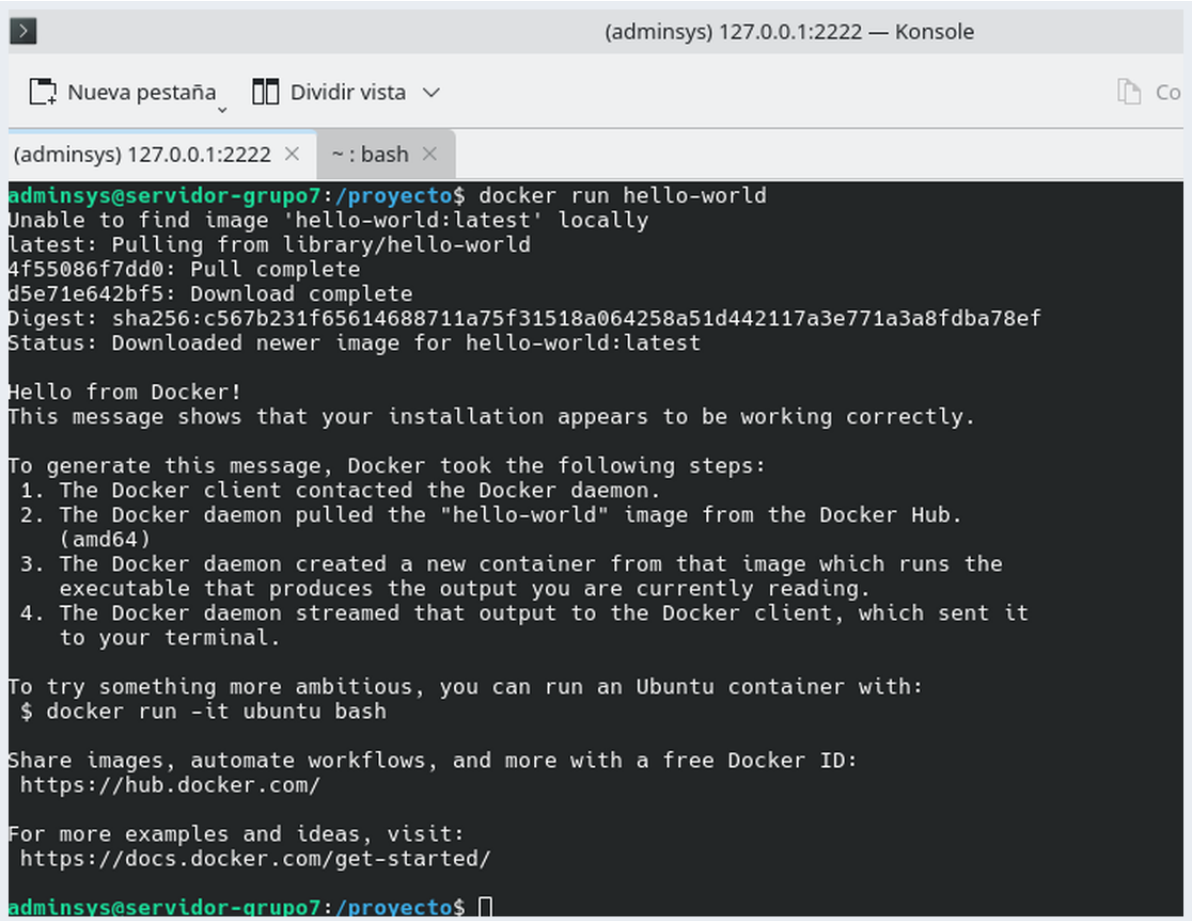
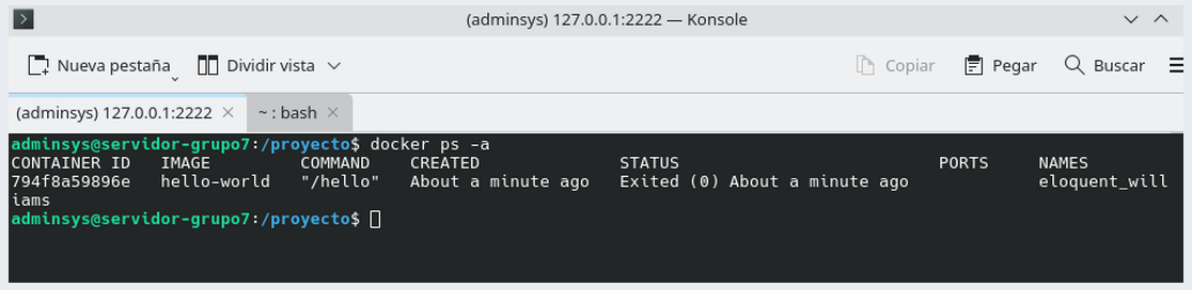


Figura 23. Contenedores con docker ps -a.



docker run hello-world: instalacion correcta



Contenedor terminado: Exited (0)

Figura 24. Prueba oficial hello-world de Docker.

Conclusión de la fase

Docker funciona y los usuarios del grupo docker pueden administrarlo sin sudo.

Fase 05 — Servidor web Nginx básico

Objetivo

Publicar el sitio del proyecto en el puerto 8080 con un bind mount.

Desarrollo

Se creó el sitio en /proyecto/web/index.html, se ejecutó nginx-basico en segundo plano con -d, se mapeó el puerto 8080 del host al 80 del contenedor y se montó /proyecto/web en /usr/share/nginx/html en modo solo lectura. Se verificó la respuesta HTTP y se revisaron los logs.

Comandos principales

```
docker run -d --name nginx-basico --restart unless-stopped \
  -p 8080:80 -v /proyecto/web:/usr/share/nginx/html:ro nginx:latest
curl -I http://localhost:8080
docker logs --tail 20 nginx-basico
docker inspect -f '{range .Mounts}{.Source} -> {.Destination}{"\n"}{end}' nginx-basico
```

Evidencias

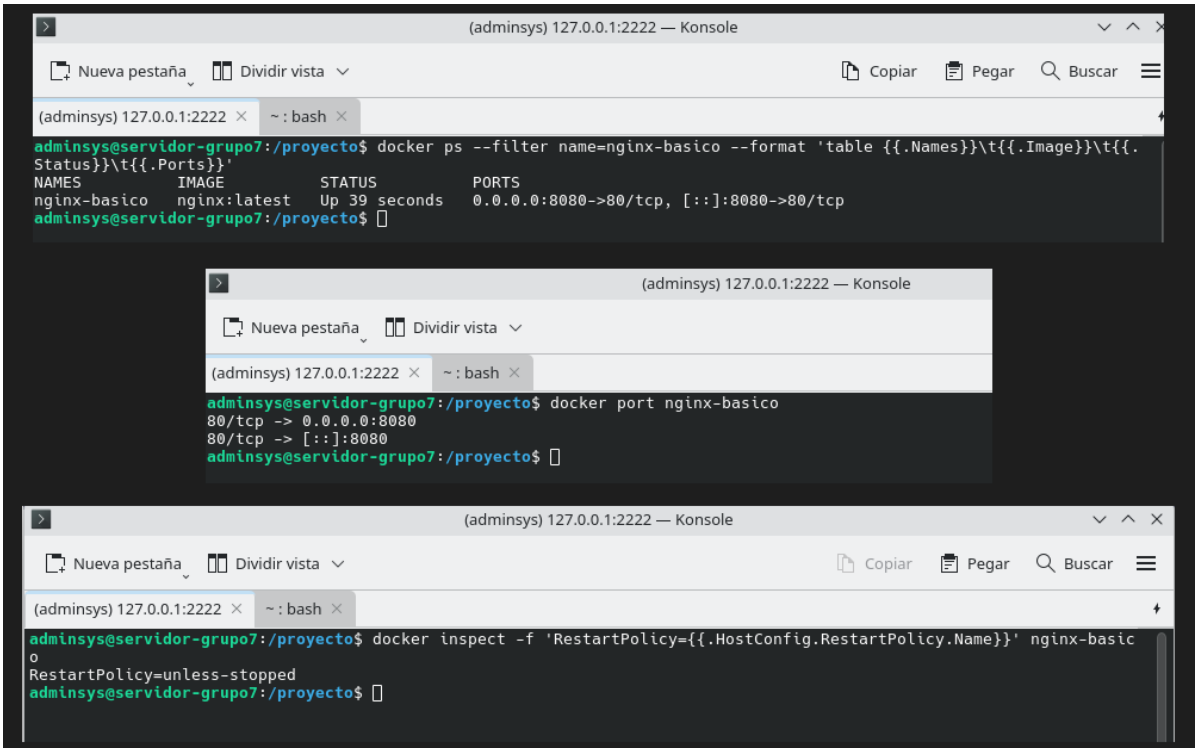


Figura 25. Contenedor nginx-basico en ejecución.

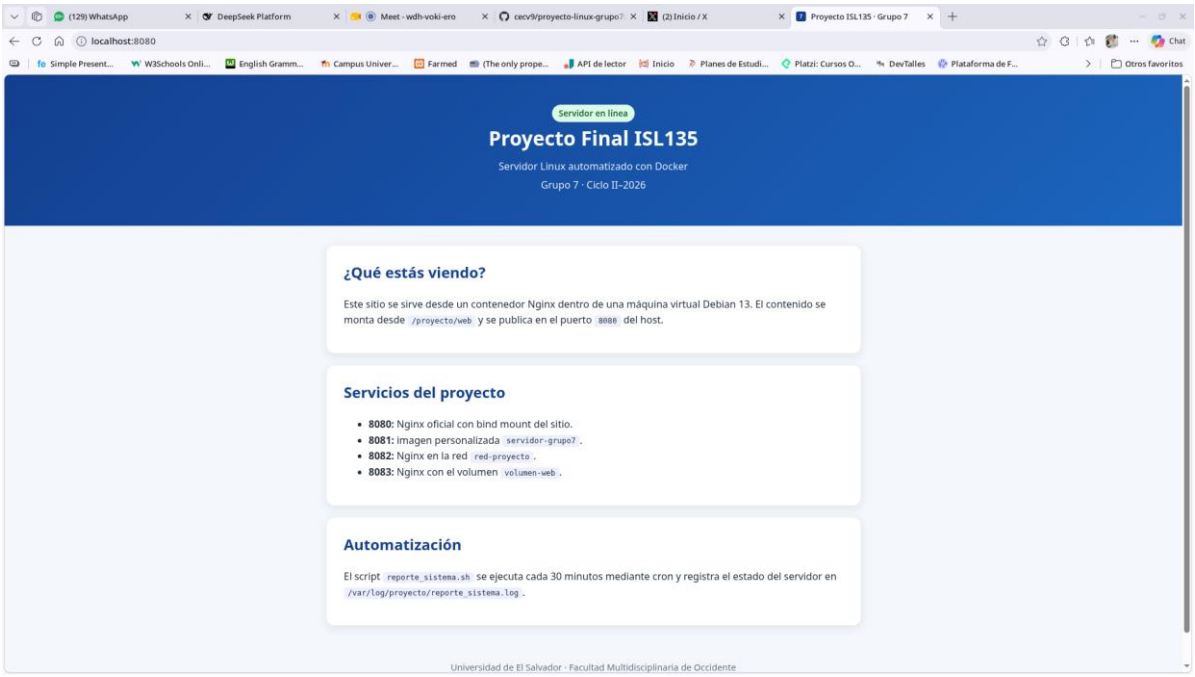


Figura 26. Sitio del proyecto visible en http://localhost:8080.

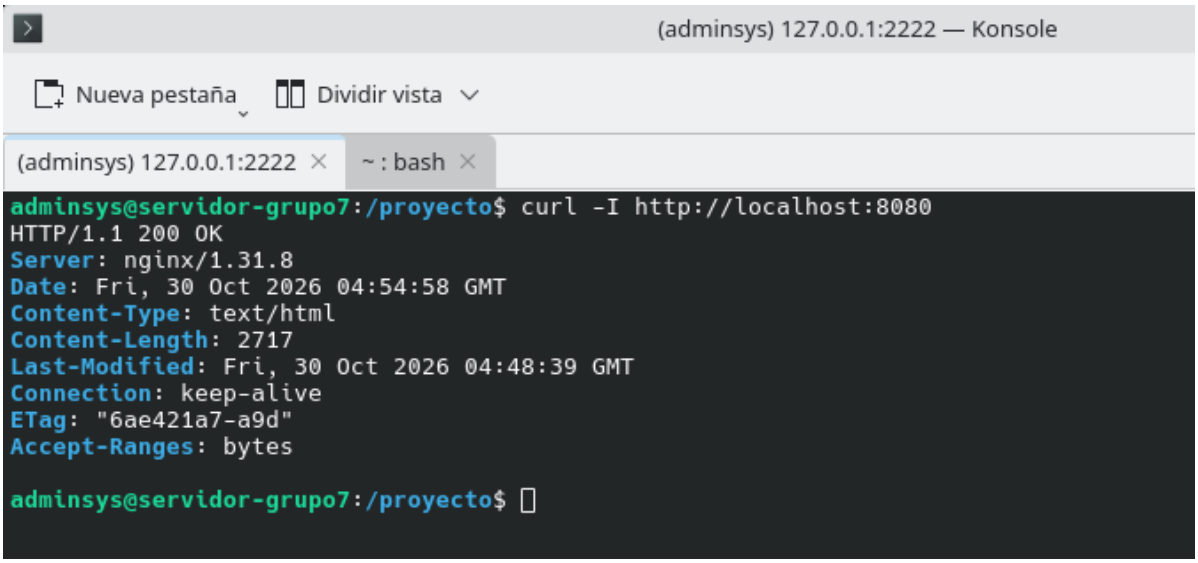


Figura 27. HTTP.

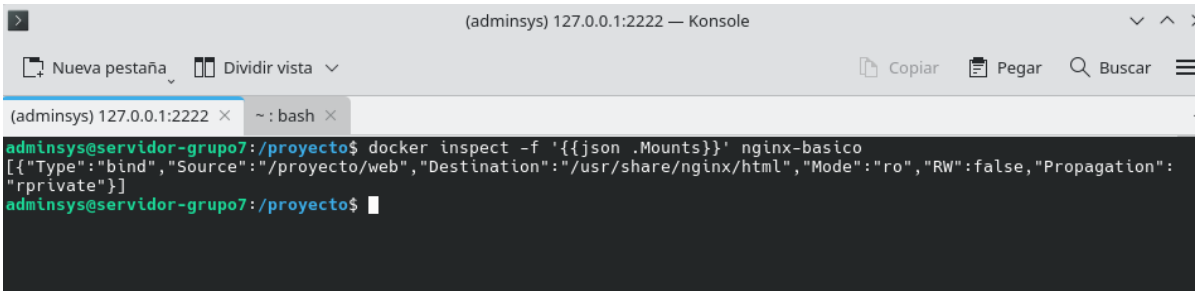


Figura 28. Montaje del contenedor.

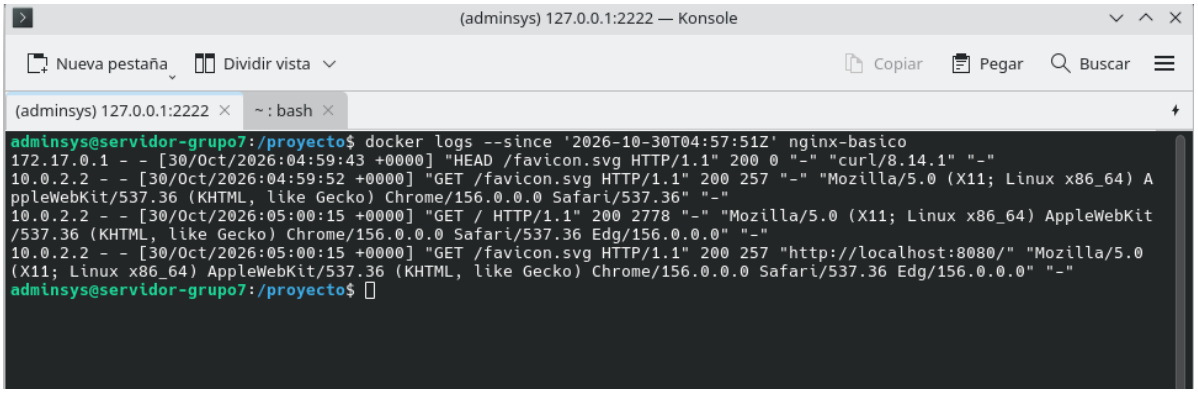


Figura 29. Logs sin errores.

Conclusión de la fase

El sitio se sirve desde el host y los cambios en /proyecto/web se reflejan sin reconstruir la imagen.

Fase 06 — Imagen personalizada con Dockerfile

Objetivo

Construir una imagen propia y compararla con la imagen oficial de Nginx.

Desarrollo

Se creó el Dockerfile con FROM nginx:latest, COPY web/ /usr/share/nginx/html/, EXPOSE 80 y CMD ["nginx", "-g", "daemon off;"]. Se construyó la imagen servidor-grupo7, se ejecutó nginx-personalizado en el puerto 8081 y se comparó con nginx-basico: la imagen propia lleva el contenido dentro y no depende de un montaje del host.

Comandos principales

```
cd /proyecto
docker build -t servidor-grupo7 .
docker run -d --name nginx-personalizado --restart unless-stopped \
  -p 8081:80 servidor-grupo7
curl -I http://localhost:8081
docker inspect -f '{json .Mounts}' nginx-personalizado
```

Evidencias

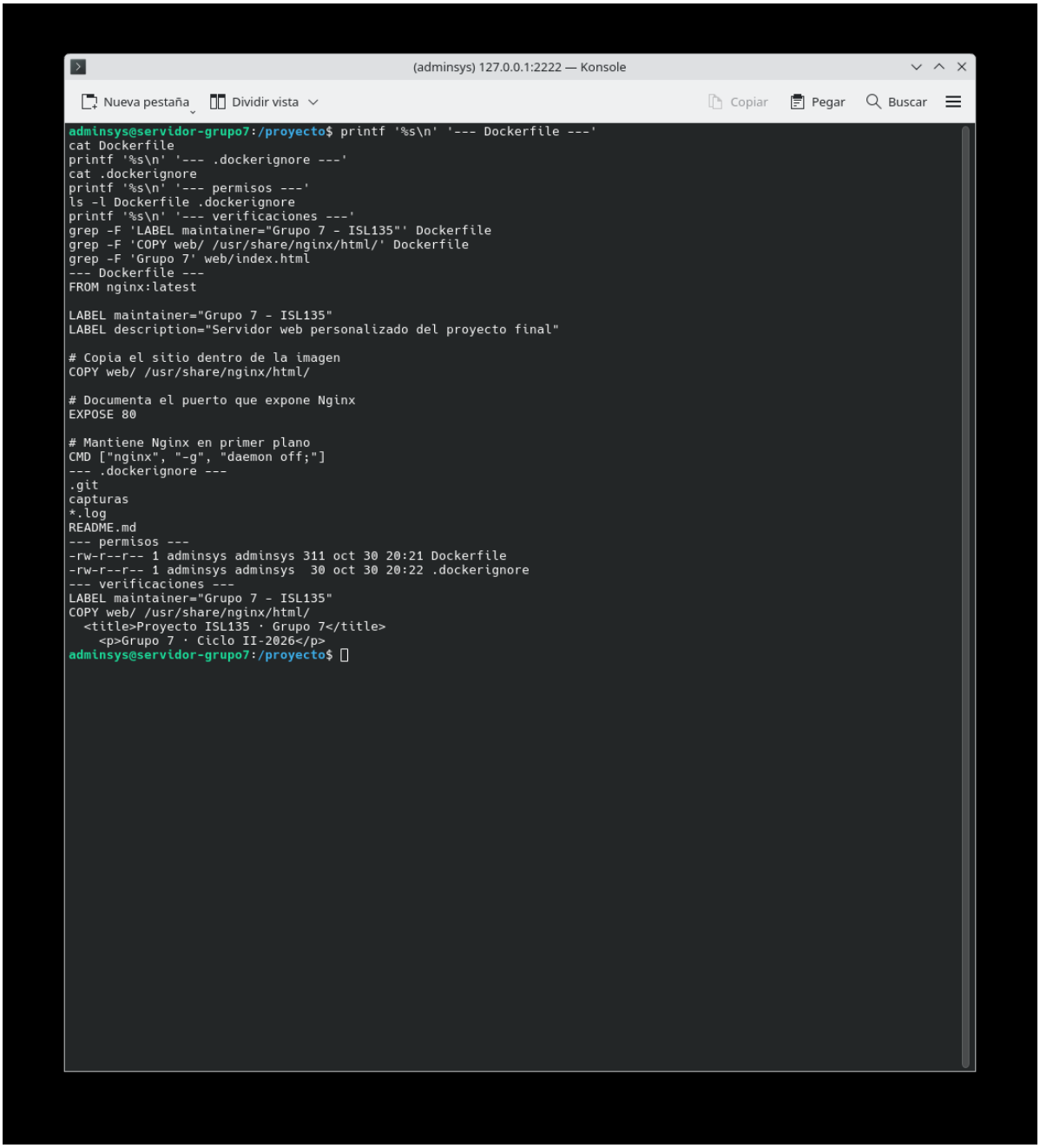


Figura 30. Dockerfile personalizado del proyecto.

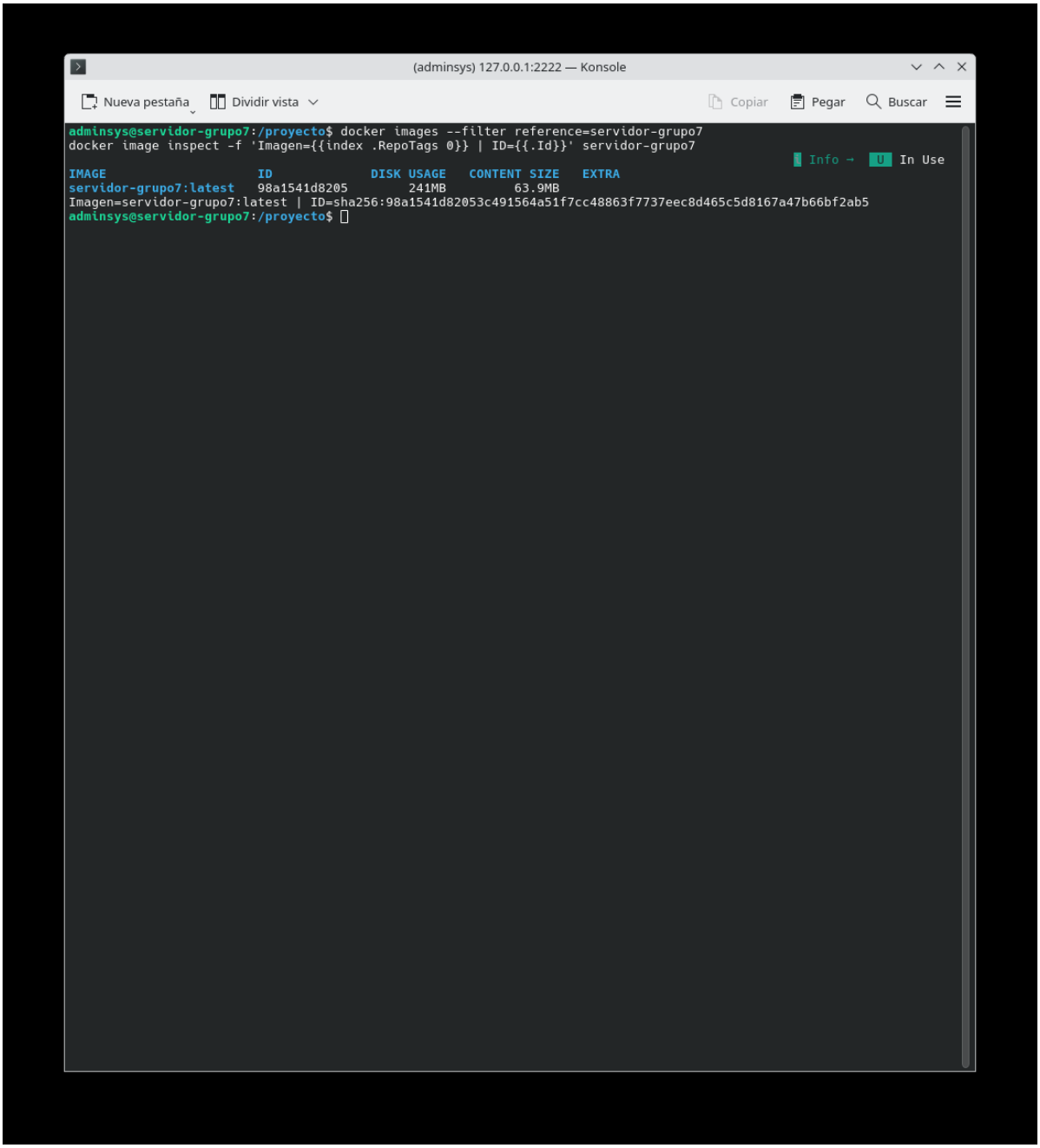


Figura 31. Construcción de la imagen servidor-grupo7.

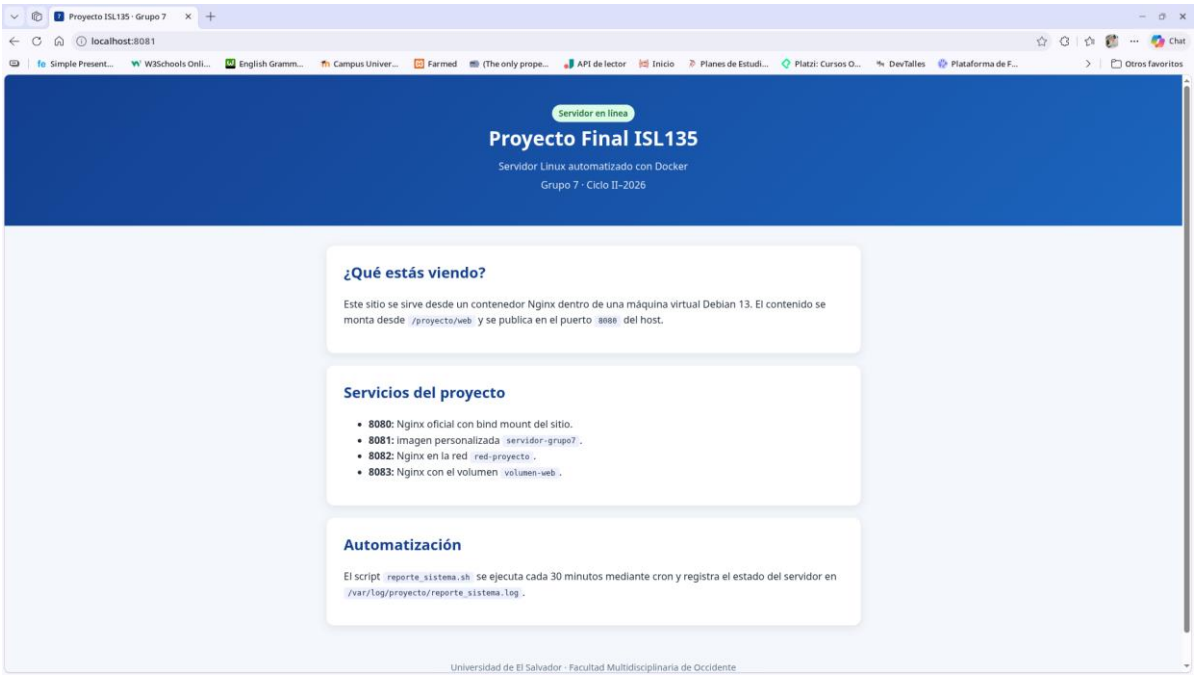


Figura 32. Sitio servido desde la imagen propia en 8081.

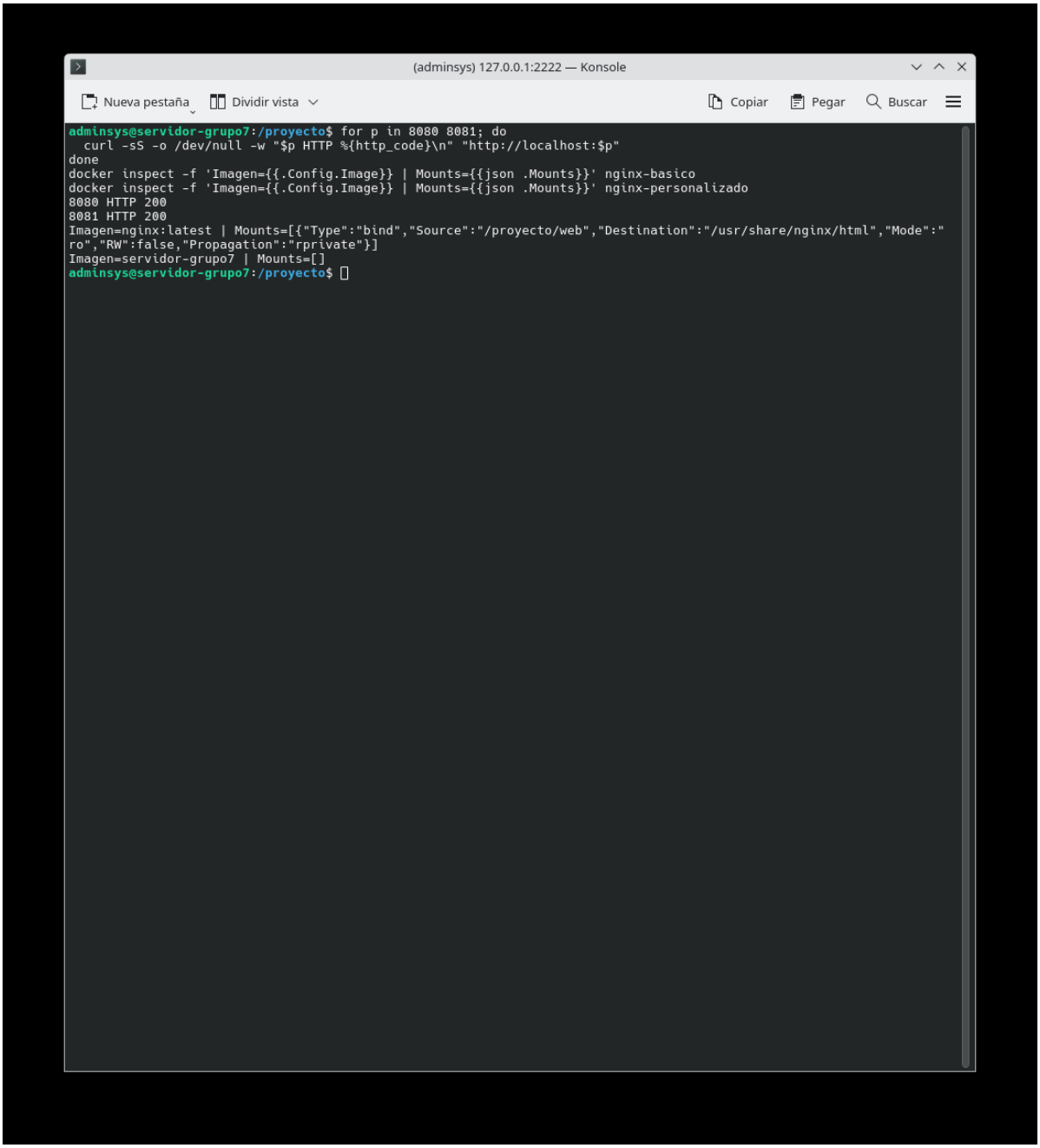


Figura 33. Comparación entre Nginx oficial y la imagen personalizada.

Conclusión de la fase

La imagen es autocontenida y reproducible; el puerto 8081 funciona sin bind mount.

Fase 07 — Redes y volúmenes de Docker

Objetivo

Aislar servicios en una red propia y conservar contenido en un volumen persistente.

Desarrollo

Se creó la red red-proyecto y el contenedor nginx-red en el puerto 8082; se comprobó con docker network inspect que está conectado. También se creó el volumen volumen-web y el contenedor nginx-volumen en el puerto 8083; se copió el sitio al volumen y se demostró que el contenido sobrevive al eliminar y recrear el contenedor.

Comandos principales

```
docker network create red-proyecto
docker run -d --name nginx-red --restart unless-stopped \
  --network red-proyecto -p 8082:80 nginx:latest
docker volume create volumen-web
docker run -d --name nginx-volumen --restart unless-stopped \
  -p 8083:80 -v volumen-web:/usr/share/nginx/html nginx:latest
docker network inspect red-proyecto
docker volume inspect volumen-web
docker cp /proyecto/web/index.html nginx-volumen:/usr/share/nginx/html/index.html
docker cp /proyecto/web/favicon.svg nginx-volumen:/usr/share/nginx/html/favicon.svg
```

Evidencias

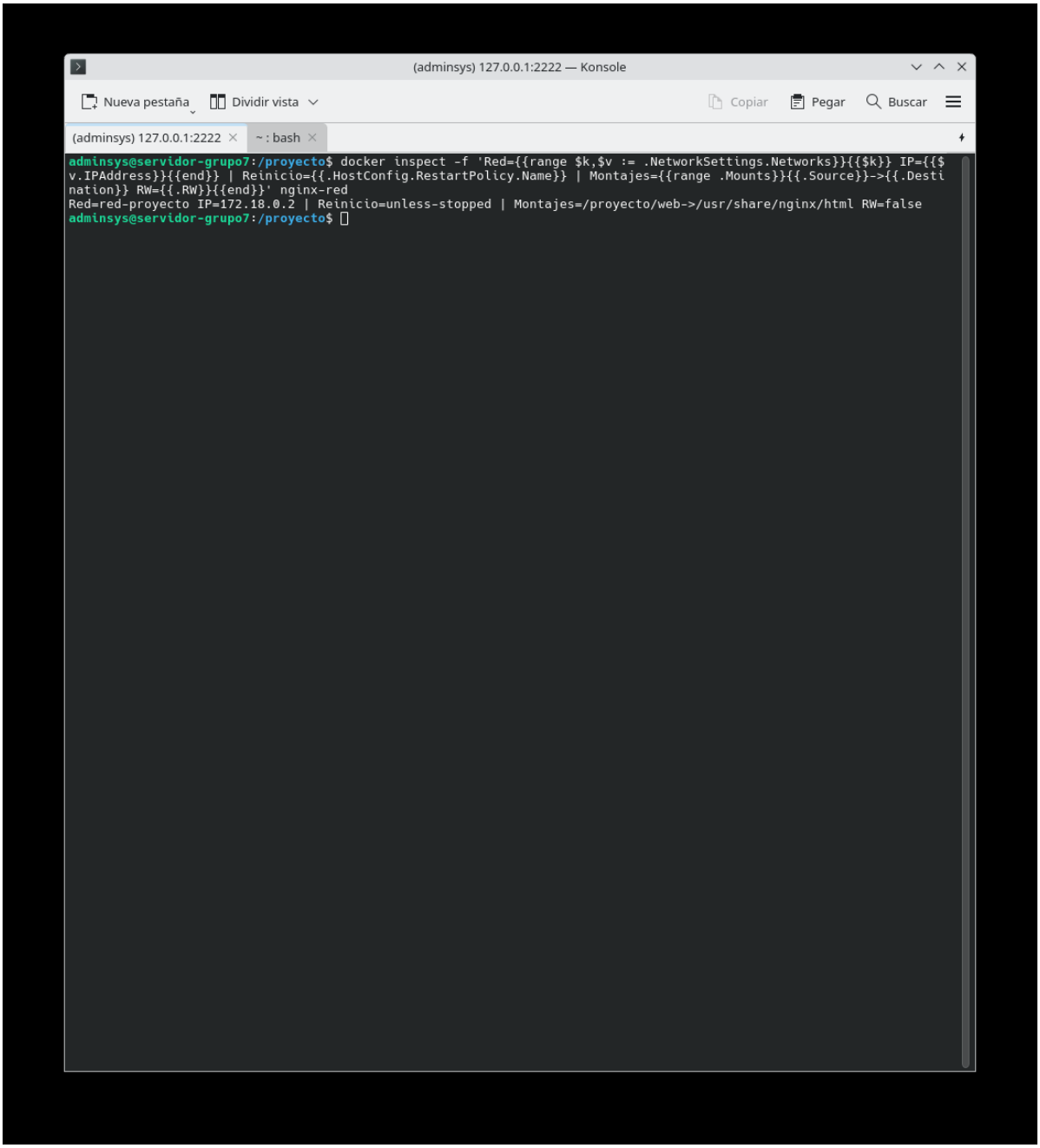


Figura 34. Inspección de nginx-red.

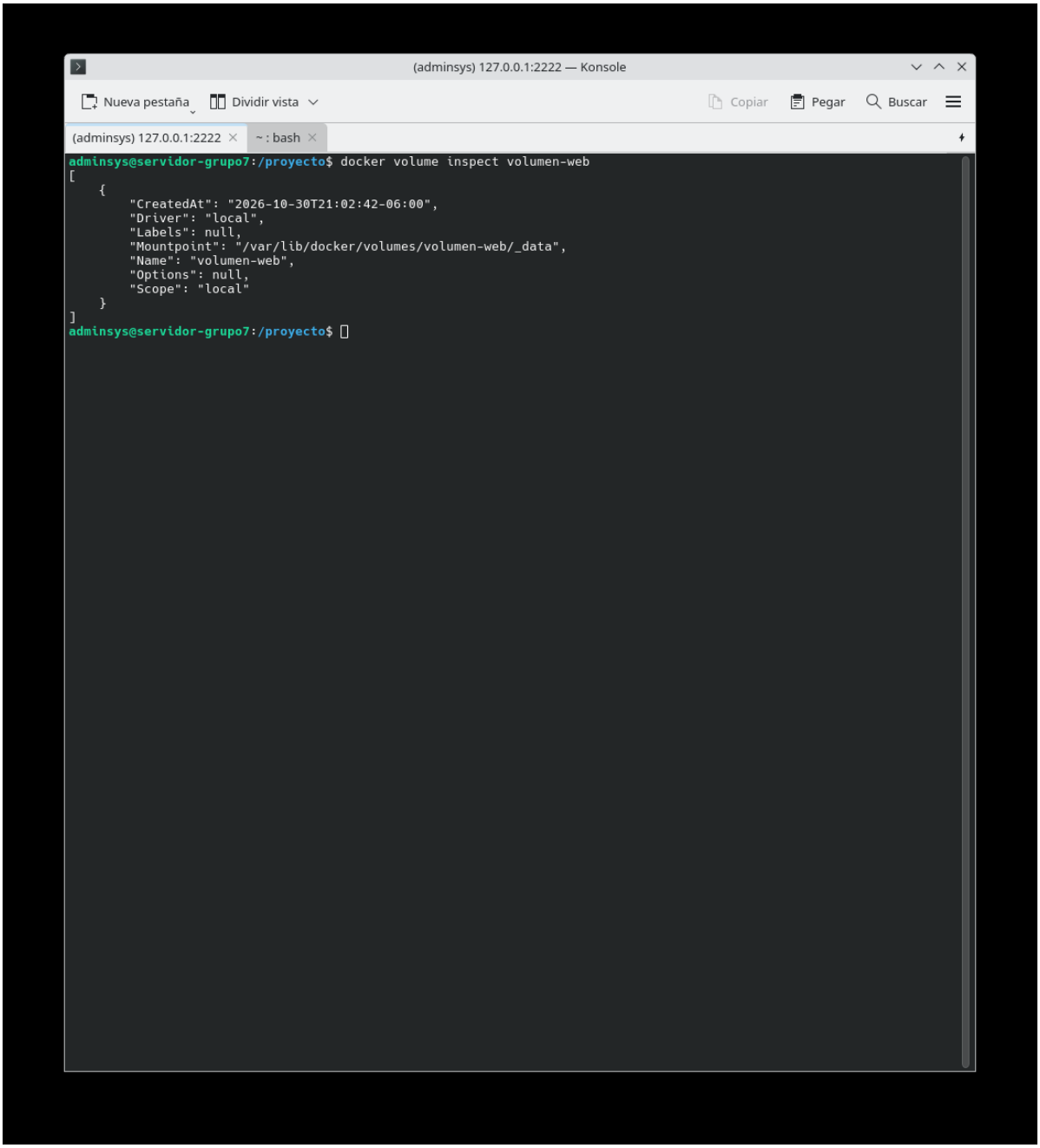


Figura 35. Inspección del volumen.

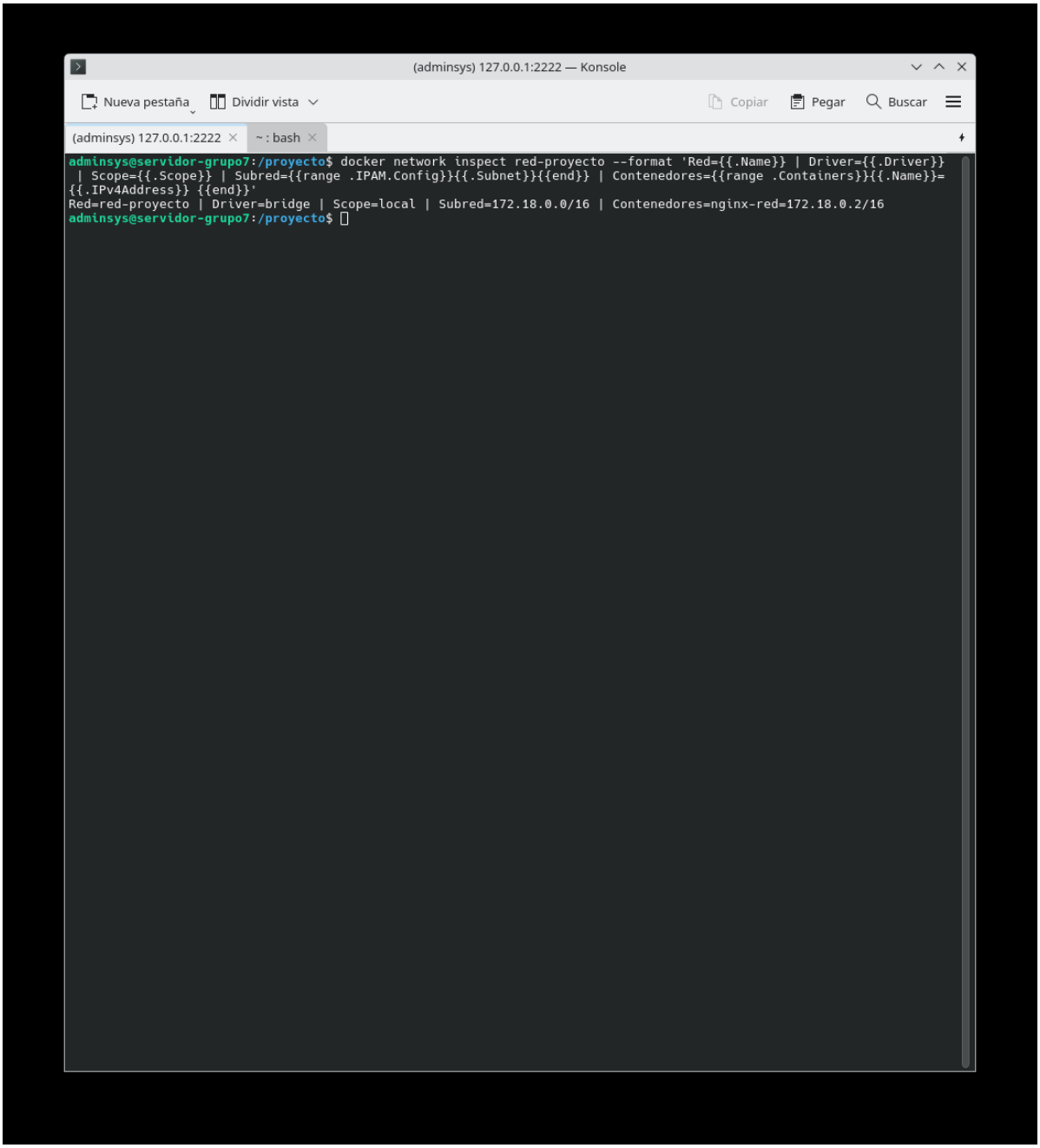


Figura 36. Red personalizada red-proyecto creada.

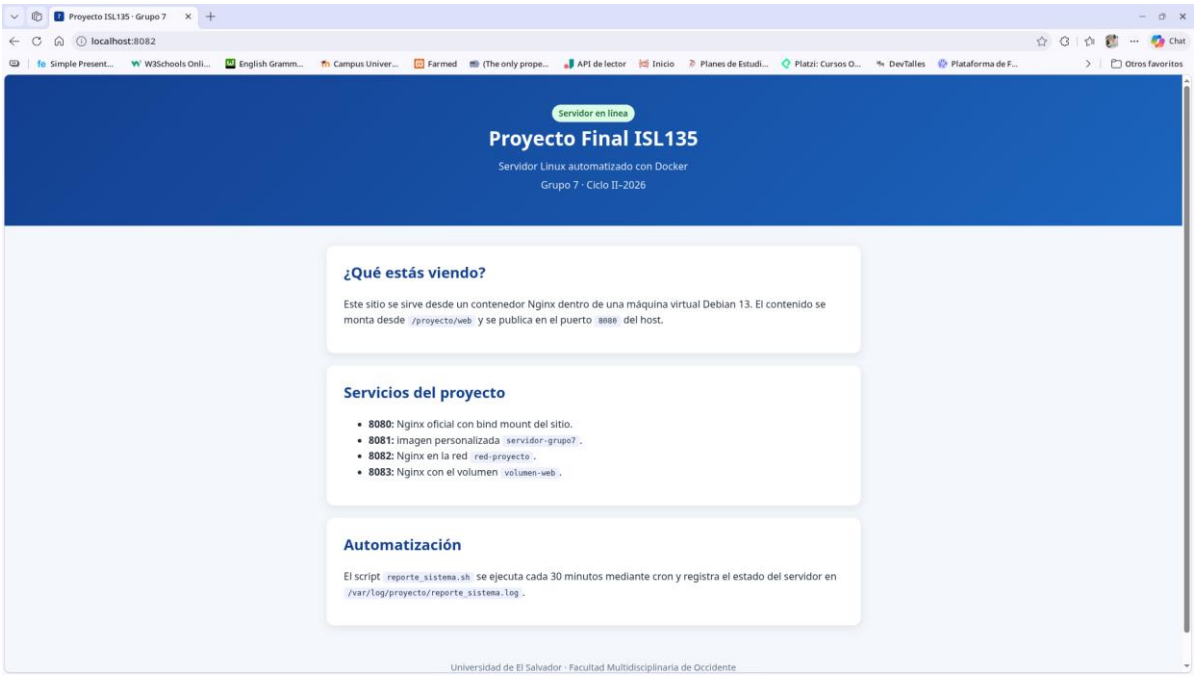
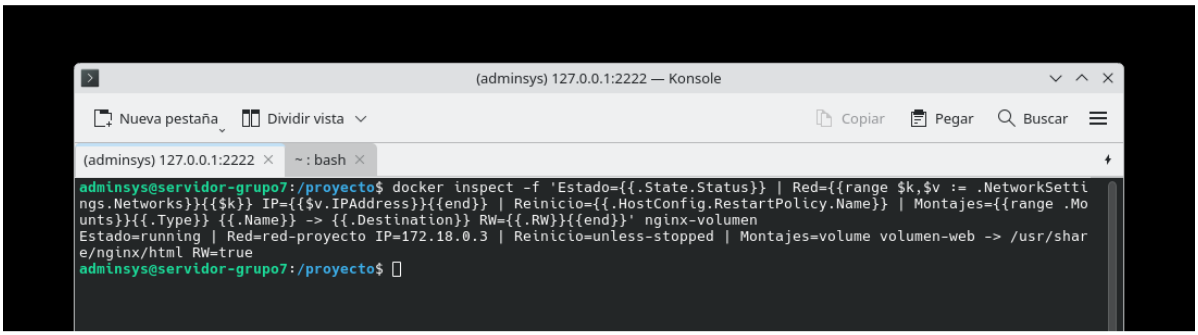


Figura 37. Sitio servido desde nginx-red en 8082.

Volumen nombrado administrado por Docker



nginx-volumen monta type=volume, name=volumen-web



Contenido del volumen: index.html y favicon.svg



Figura 38. Volumen volumen-web creado e inspeccionado.

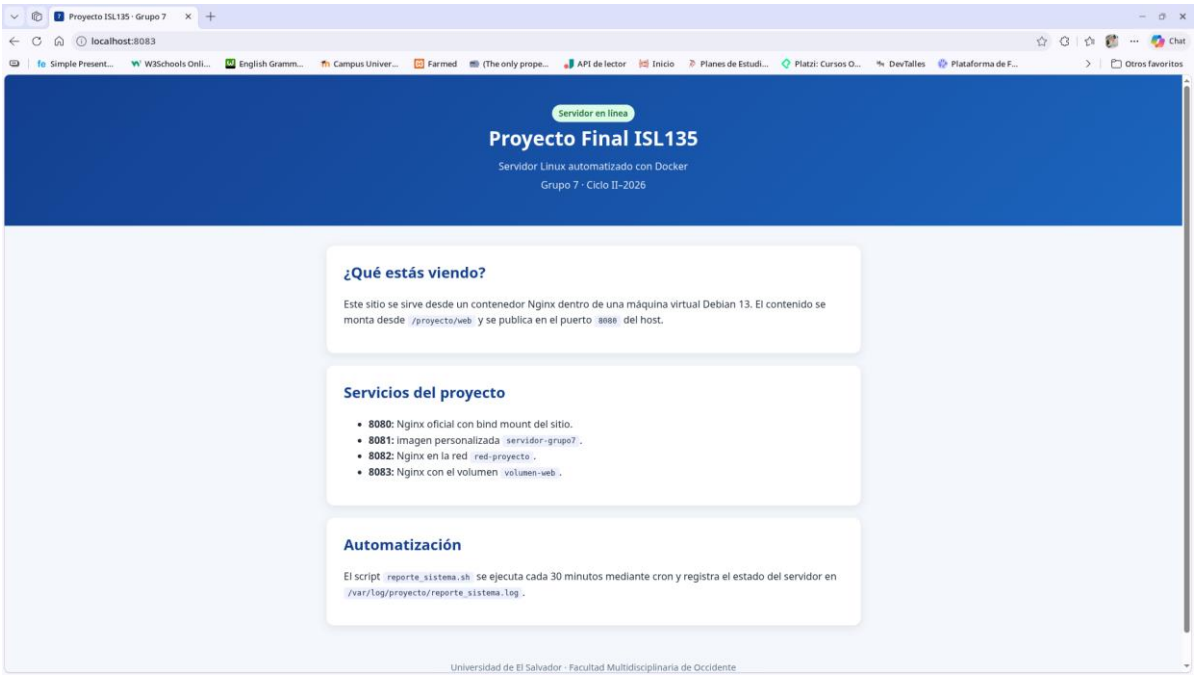
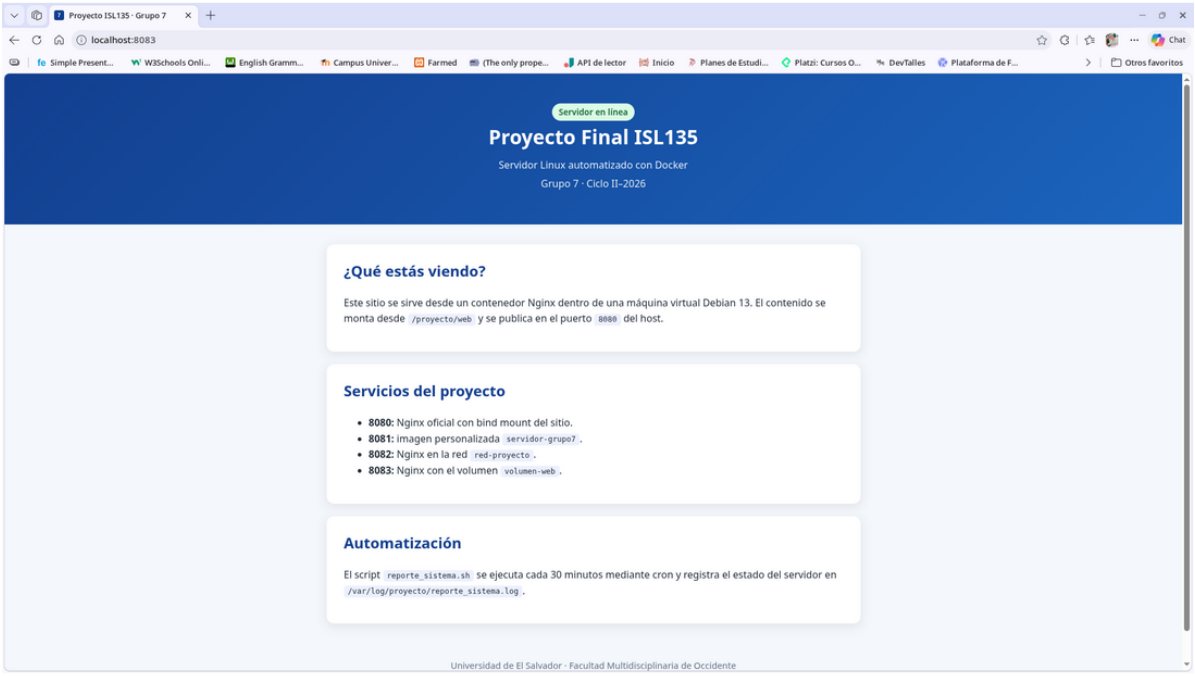
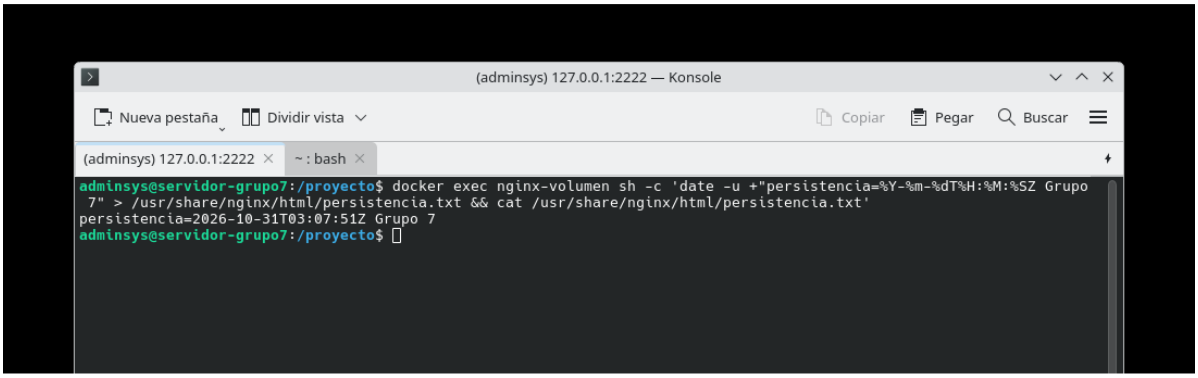


Figura 39. Sitio servido desde nginx-volumen en 8083.

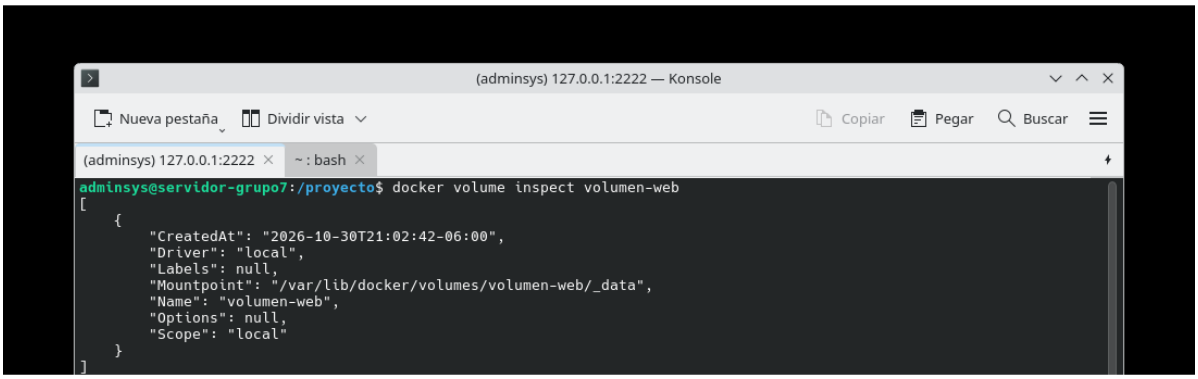
Navegador después de recrear: http://localhost:8083



Testigo creado antes de eliminar el contenedor



Volumen todavía presente sin el contenedor



Mismo testigo después de recrear el contenedor

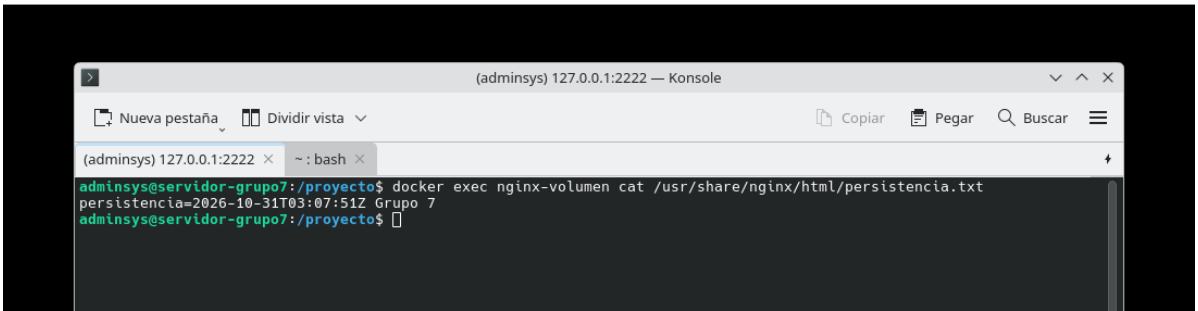


Figura 40. Persistencia del contenido después de recrear el contenedor.

Conclusión de la fase

Los contenedores se comunican por nombre y el contenido del volumen persiste aunque el contenedor se recree.

Fase 08 — Pruebas y verificación final

Objetivo

Demostrar que todos los servicios, la automatización y el repositorio funcionan juntos.

Desarrollo

Se ejecutó un contenedor Alpine en red-proyecto para hacer ping a nginx-red por nombre, se comprobaron los 4 puertos con curl, se revisaron cron y el log, y se ejecutó verificar_proyecto.sh, que valida hostname, usuarios, grupos, permisos, script, cron, Docker, contenedores, red, volumen y Git. También se reinició la VM y se confirmó que los servicios vuelven a arrancar.

Comandos principales

```
docker run --rm --network red-proyecto alpine ping -c 2 nginx-red
for p in 8080 8081 8082 8083; do
  curl -s -o /dev/null -w "$p -> %{http_code}\n" http://localhost:$p
done
/proyecto/scripts/verificar_proyecto.sh; echo "Código de salida: $?"
git -C /proyecto status -sb
sudo reboot
```

Evidencias

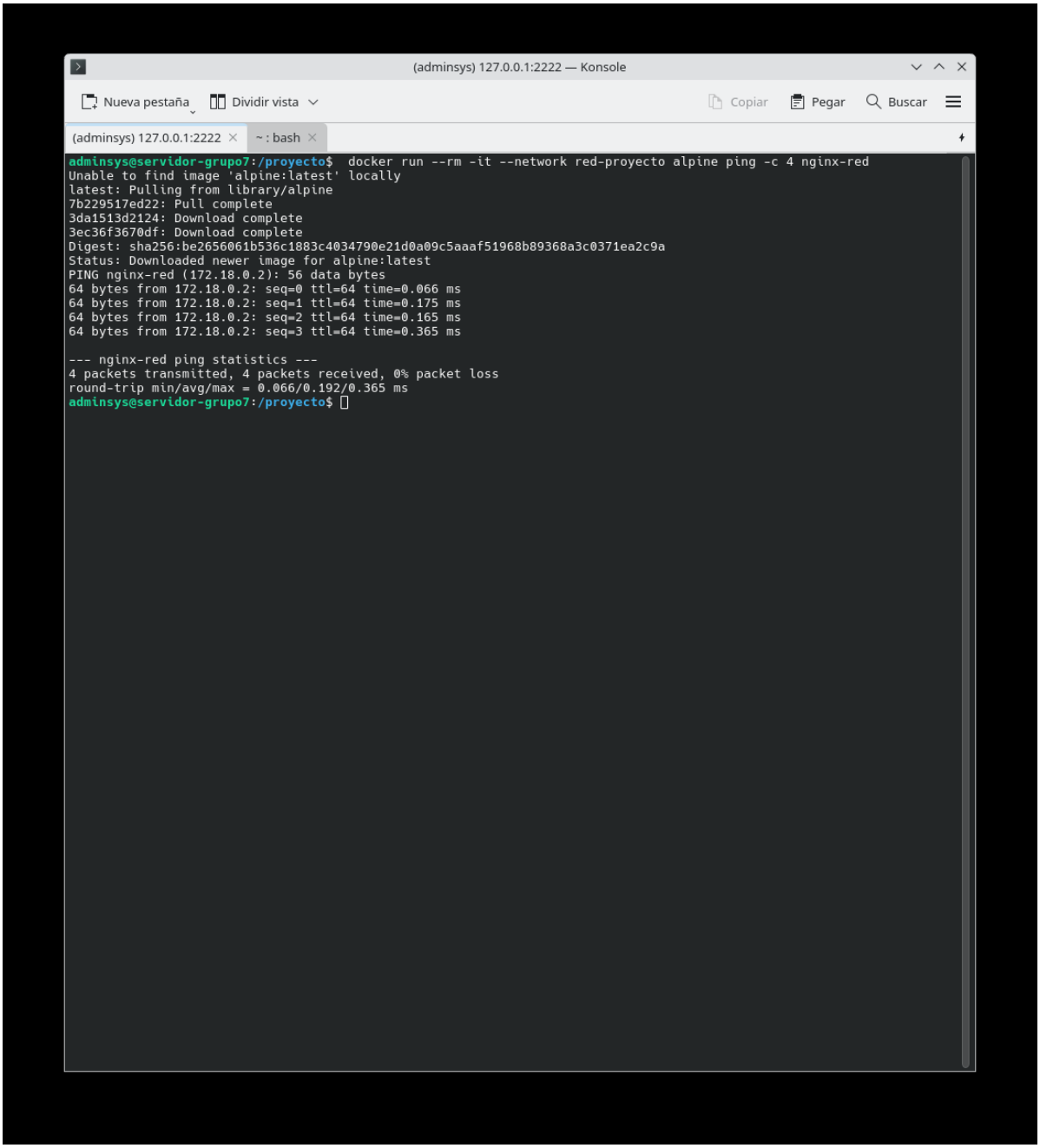


Figura 41. Ping desde Alpine a nginx-red usando la red personalizada.

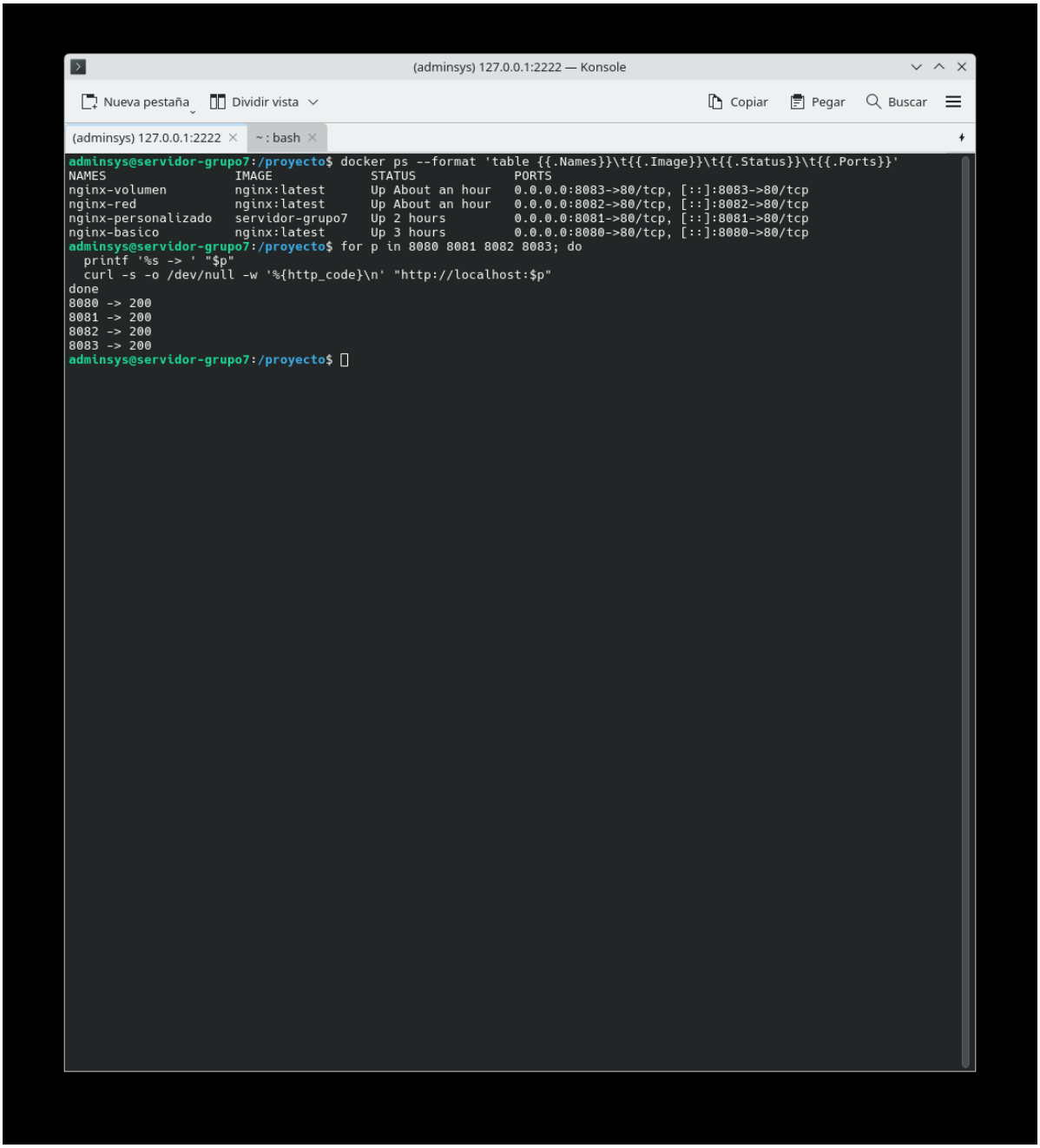


Figura 42. Los cuatro puertos responden HTTP 200.

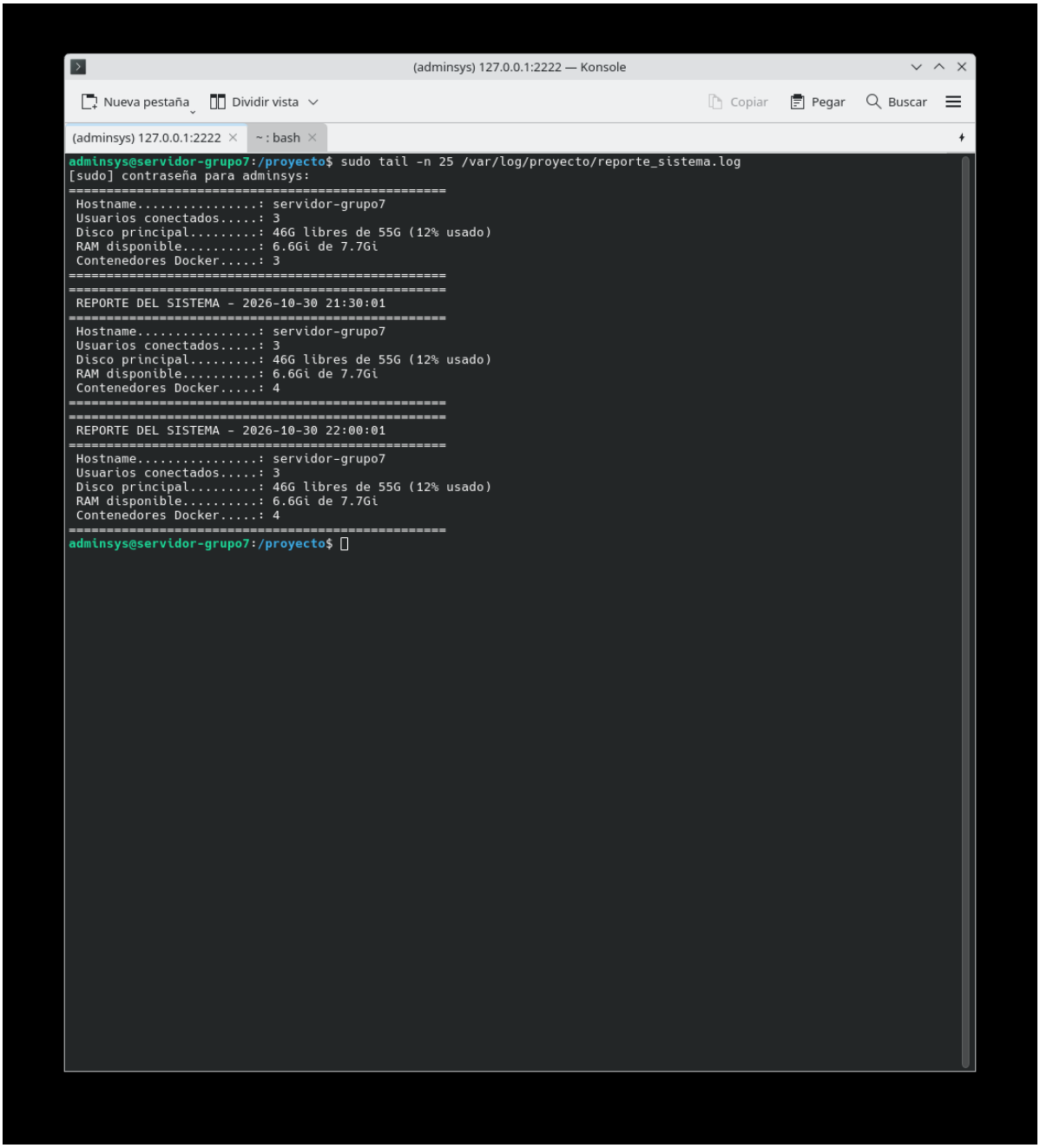


Figura 43. Cron activo y log actualizado.

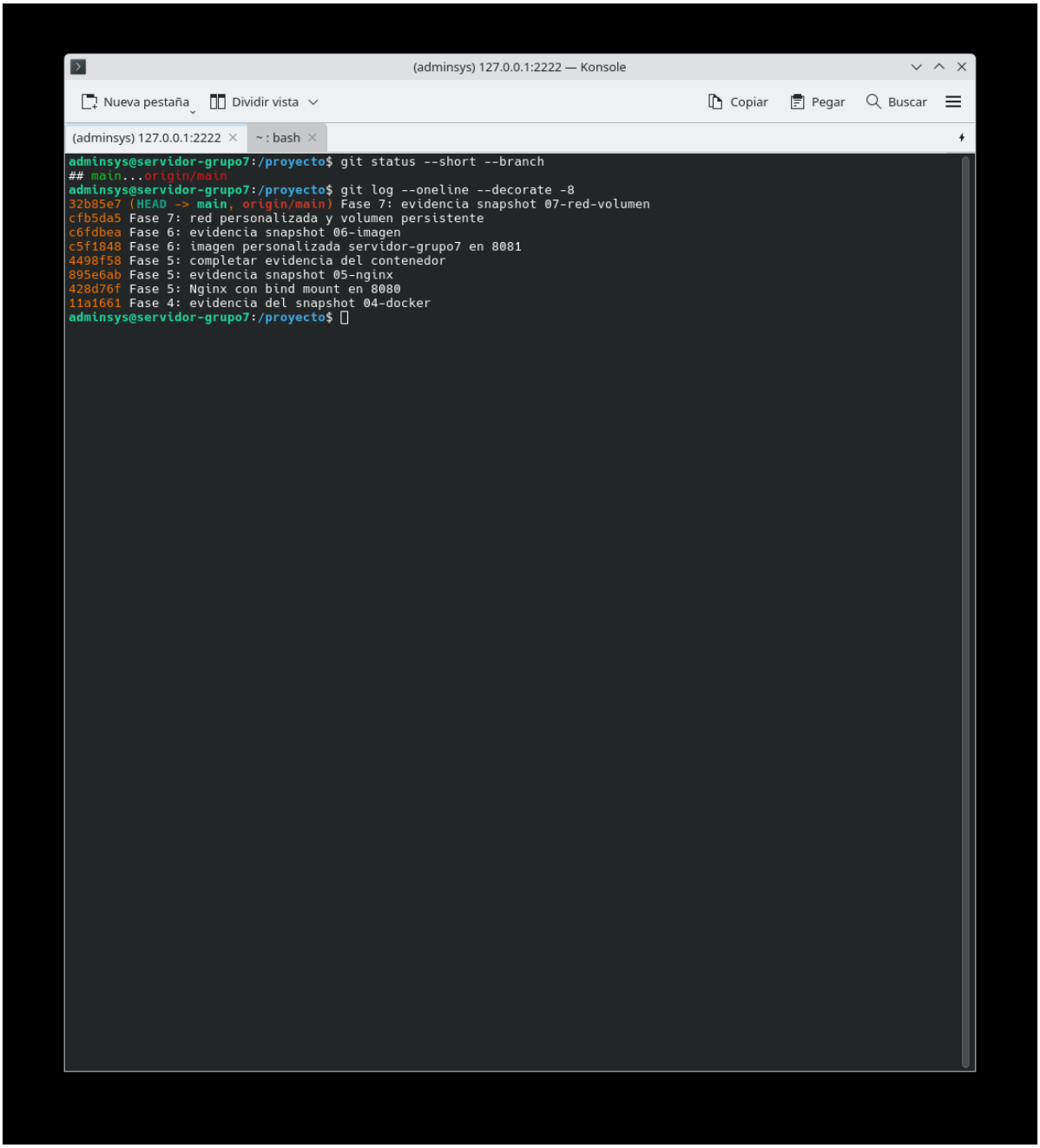


Figura 44. Git status limpio y log de commits.

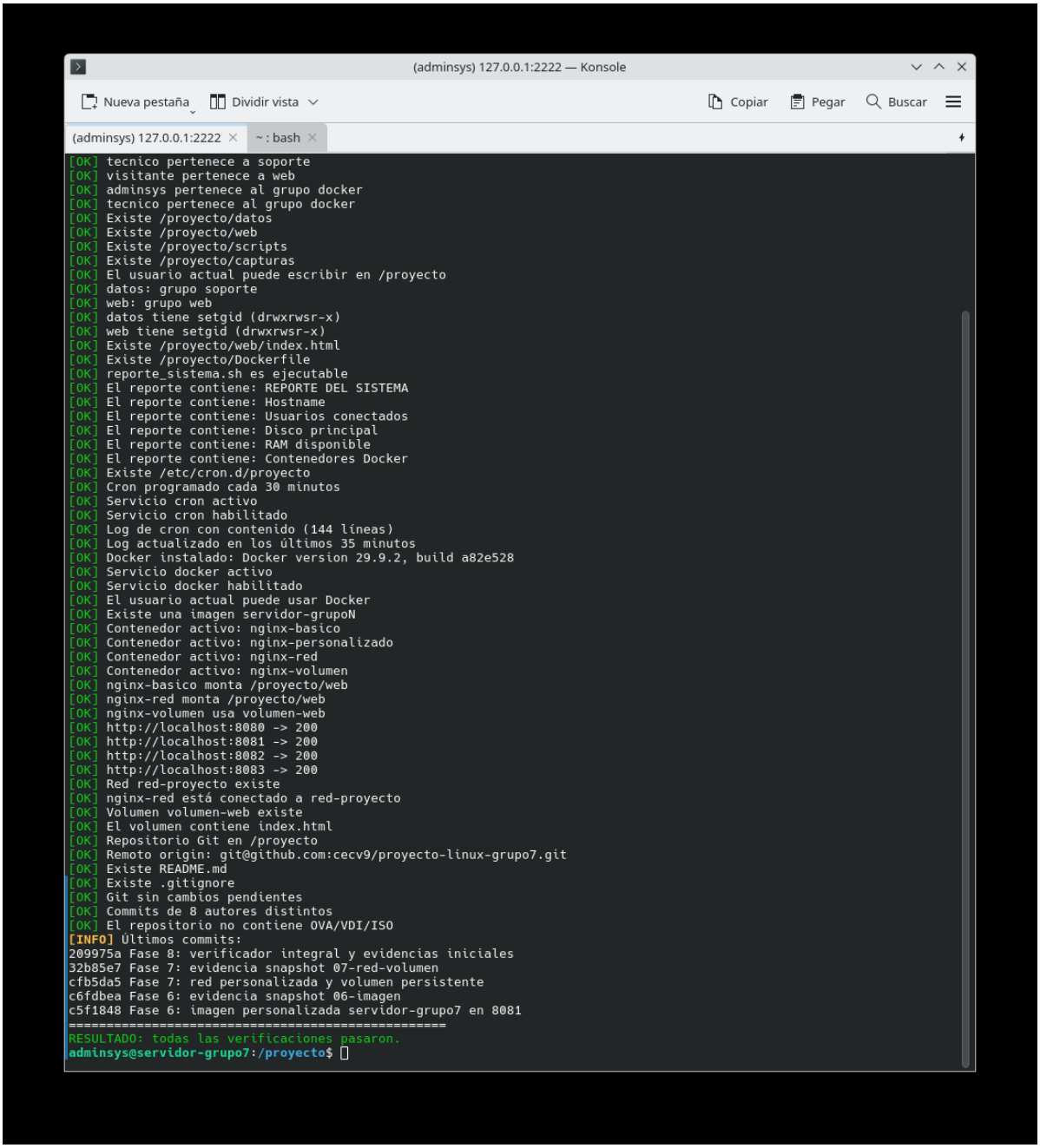


Figura 45. Verificación integral aprobada.

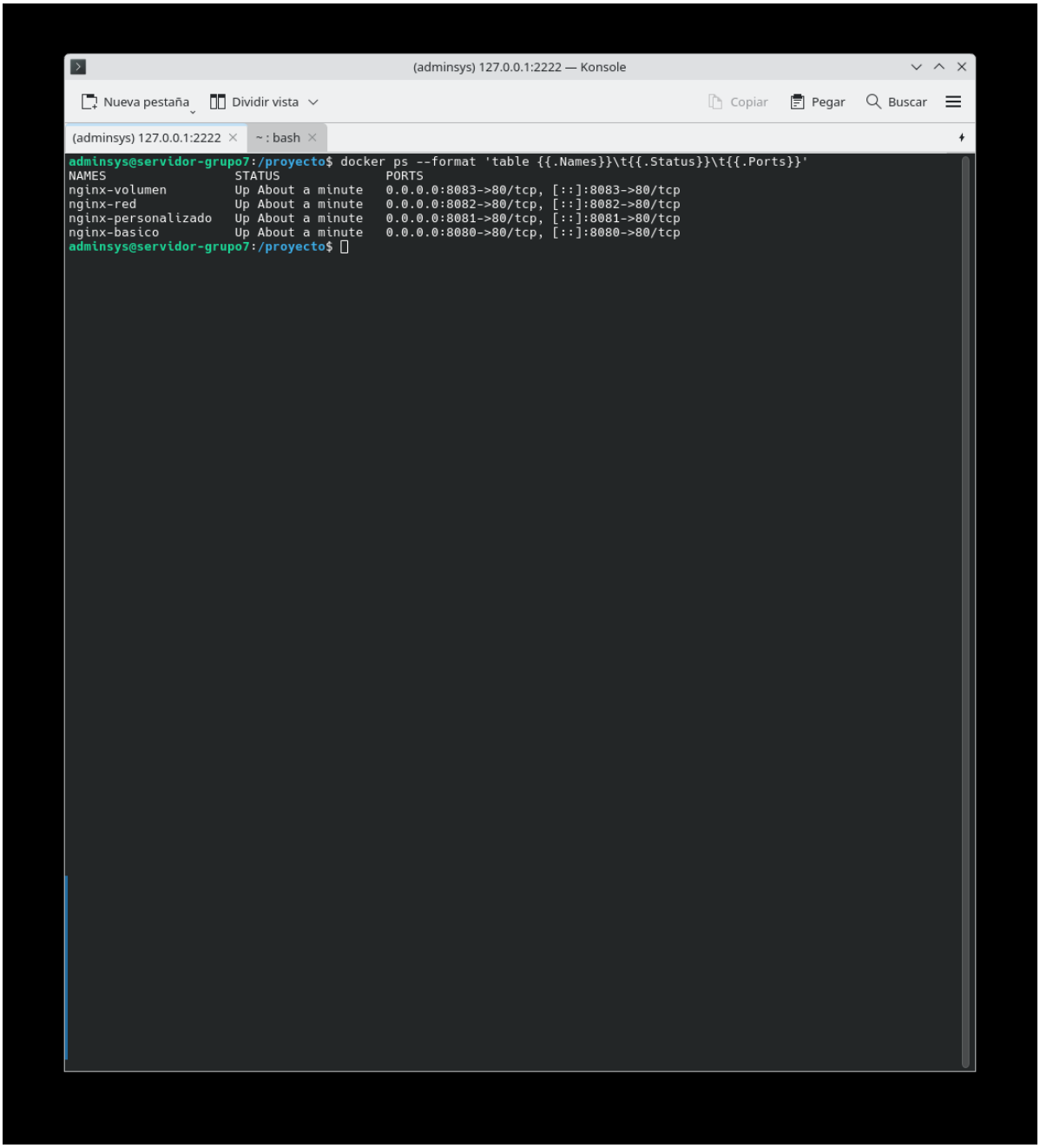


Figura 46. Los contenedores vuelven a arrancar después del reinicio.

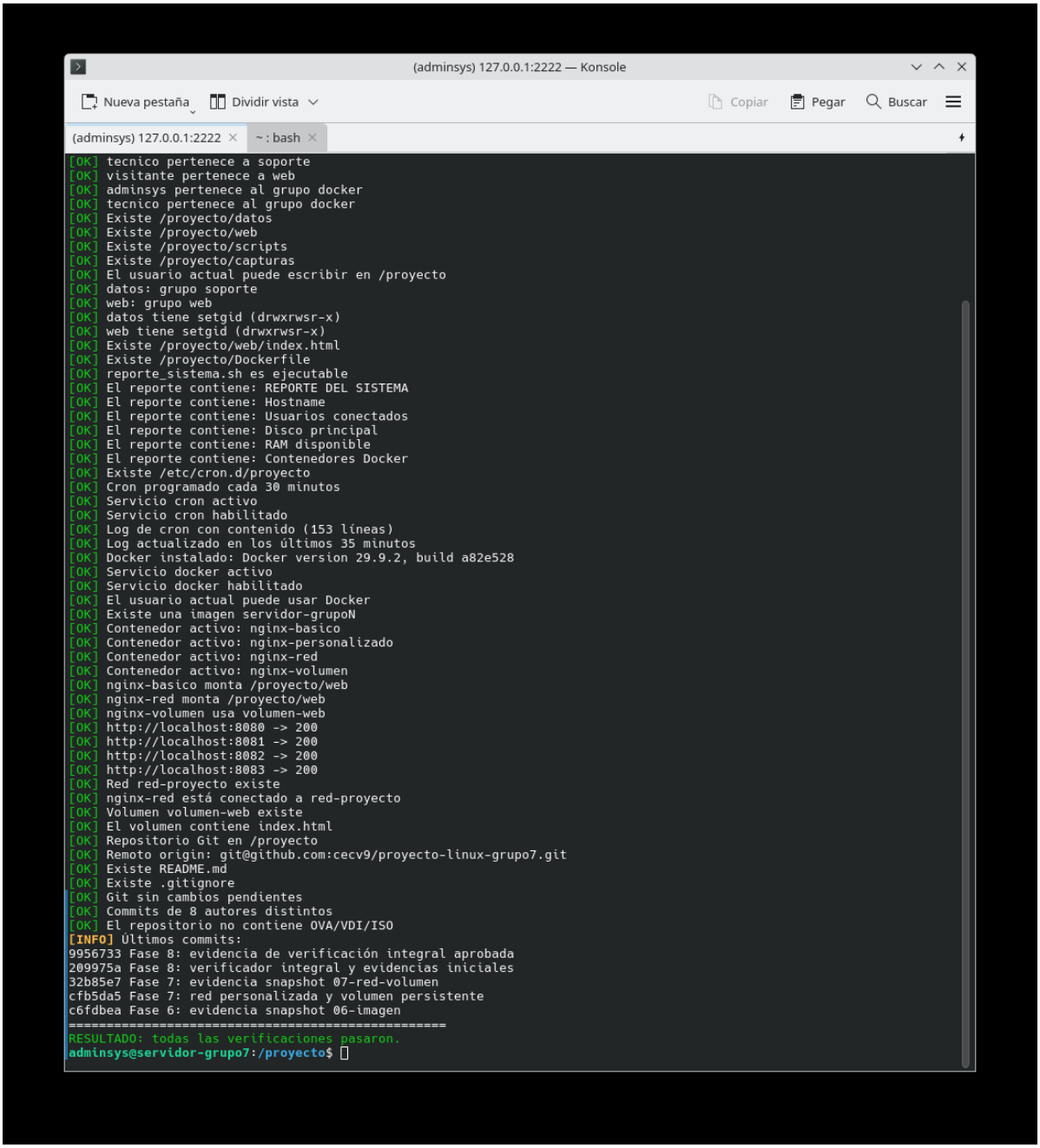


Figura 47. Verificación integral posterior al reinicio.

Conclusión de la fase

El proyecto queda verificado de extremo a extremo y resiste un reinicio de la máquina virtual.

Exportación e importación de la OVA

Objetivo

Demostrar que la máquina virtual se puede exportar como proyecto-isl135-grupo7.ova,

verificar su integridad con SHA-256, importarla en otra copia y comprobar que los

cuatro servicios siguen funcionando.

Desarrollo

7. Se verificaron los reenvíos NAT 2222, 8080, 8081, 8082 y 8083.
8. Se apagó la VM y se exportó a OVA con VBoxManage export.
9. Se generó y verificó el SHA-256.
10. Se importó como servidor-grupo7-copia con MAC regenerada.
11. Se comprobó que los contenedores y los cuatro puertos responden 200.

Comandos principales

```
# En el host, con la VM apagada
VBoxManage export "Debian 13" --output "proyecto-isl135-grupo7.ova" --manifest
sha256sum proyecto-isl135-grupo7.ova | tee proyecto-isl135-grupo7.ova.sha256
sha256sum -c proyecto-isl135-grupo7.ova.sha256

# Importar como copia independiente
VBoxManage import "proyecto-isl135-grupo7.ova" --vsys 0 --vmname "servidor-grupo7-copia"
VBoxManage modifyvm "servidor-grupo7-copia" --macaddress1 auto
VBoxManage showvminfo "servidor-grupo7-copia" --machinereadable | grep -i Forwarding
```

Evidencias

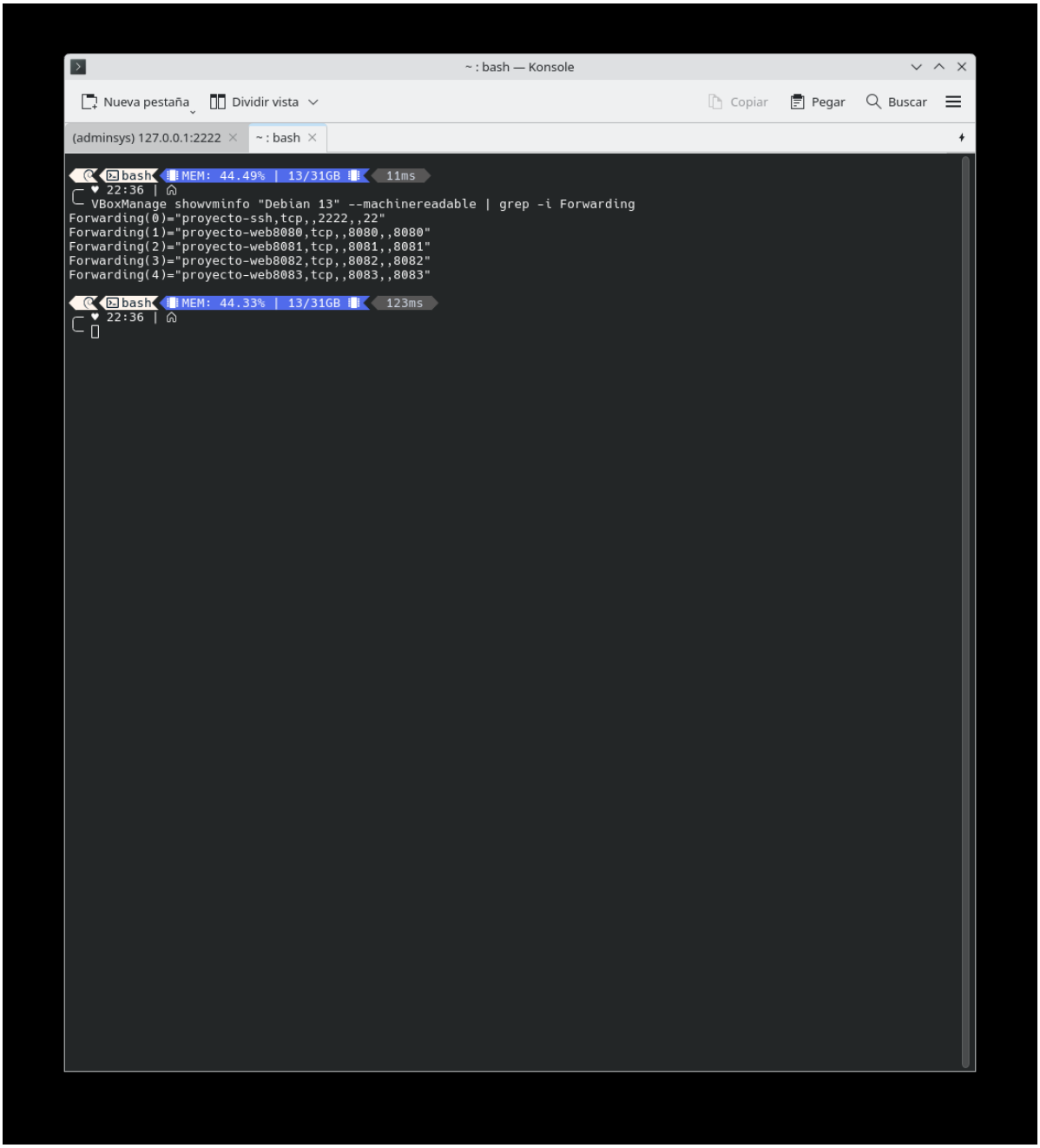


Figura 48. Reenvíos NAT configurados antes de exportar.

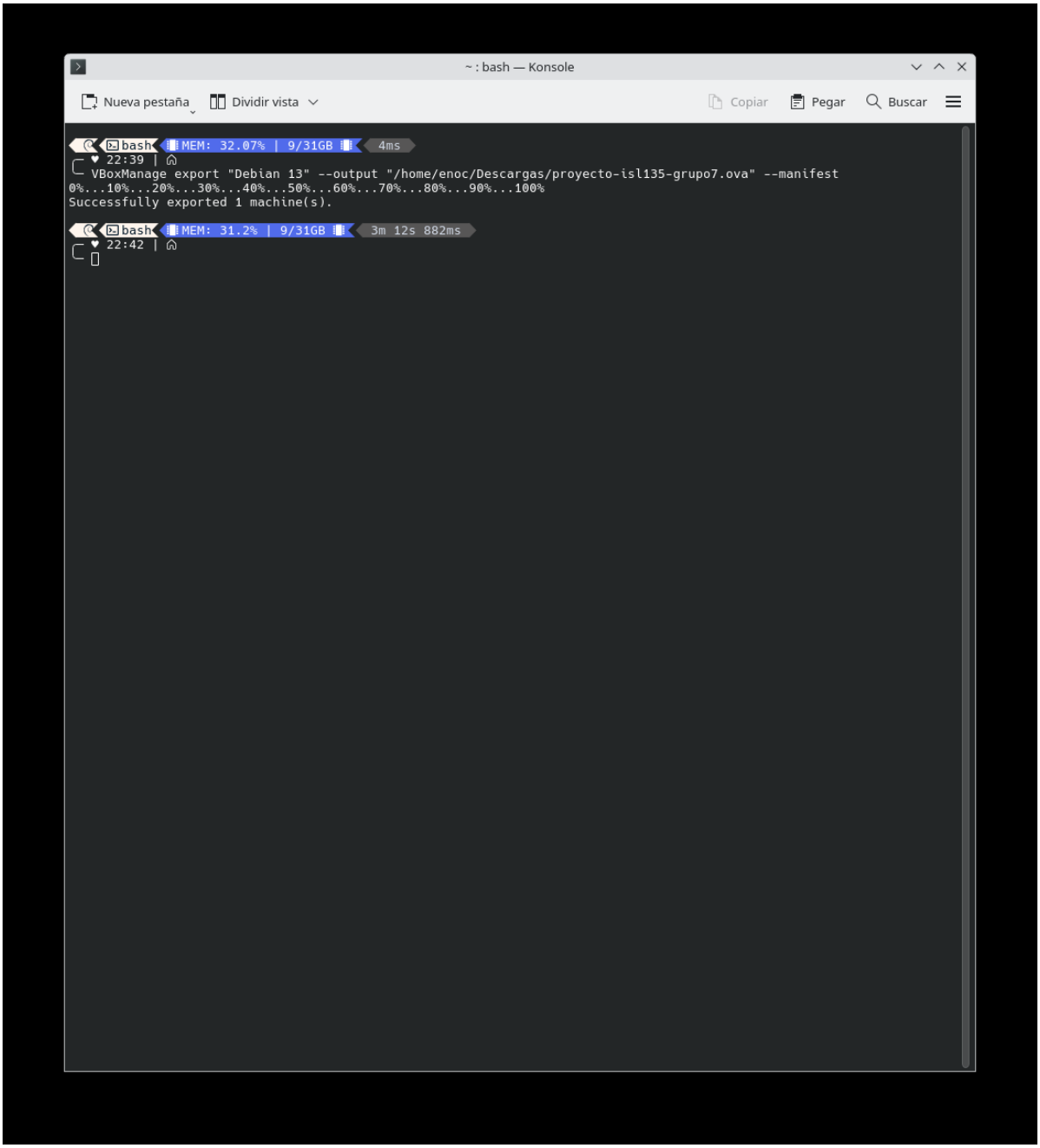


Figura 49. Exportación de la VM a OVA completada.

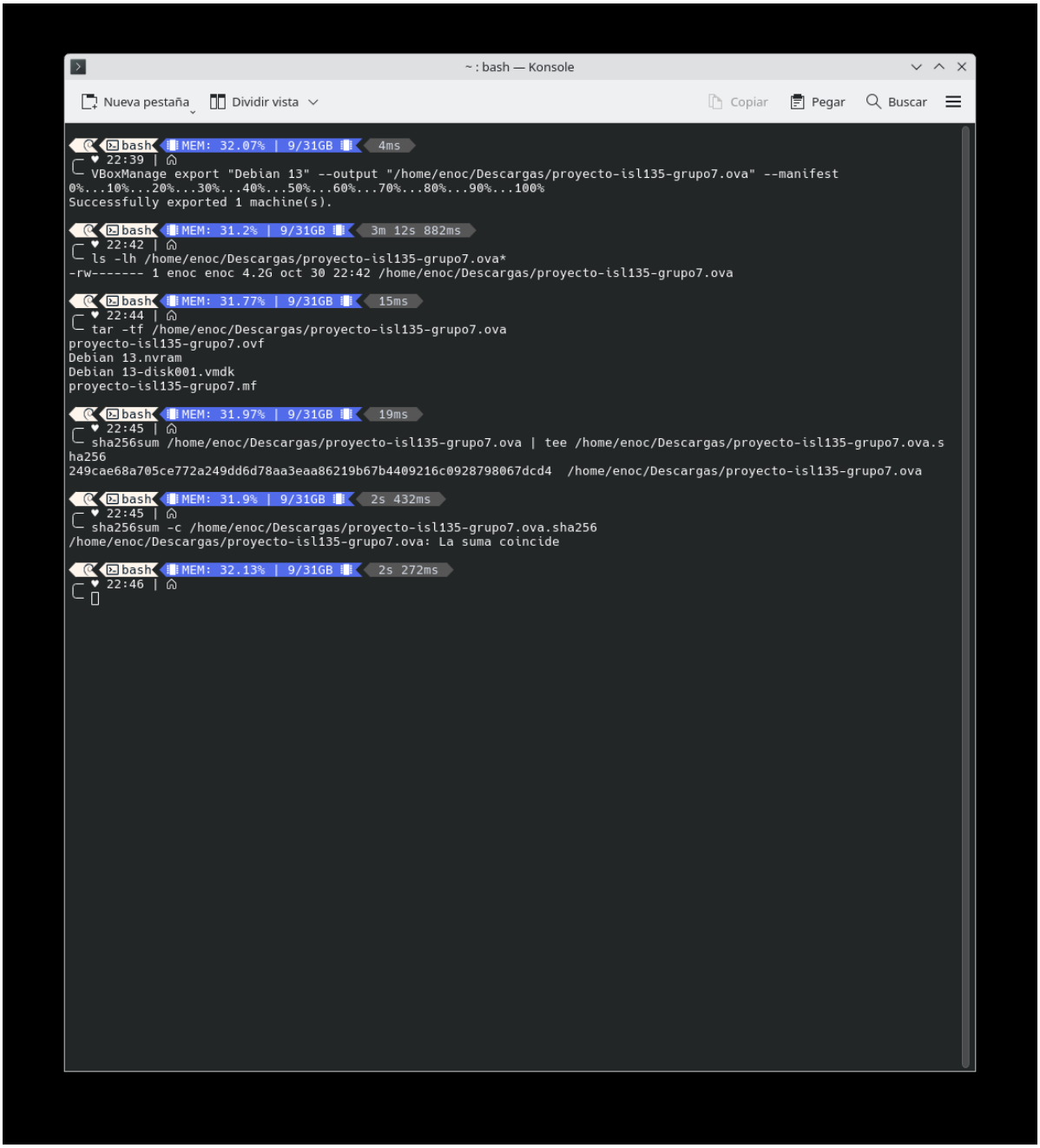


Figura 50. Verificación del checksum SHA-256.

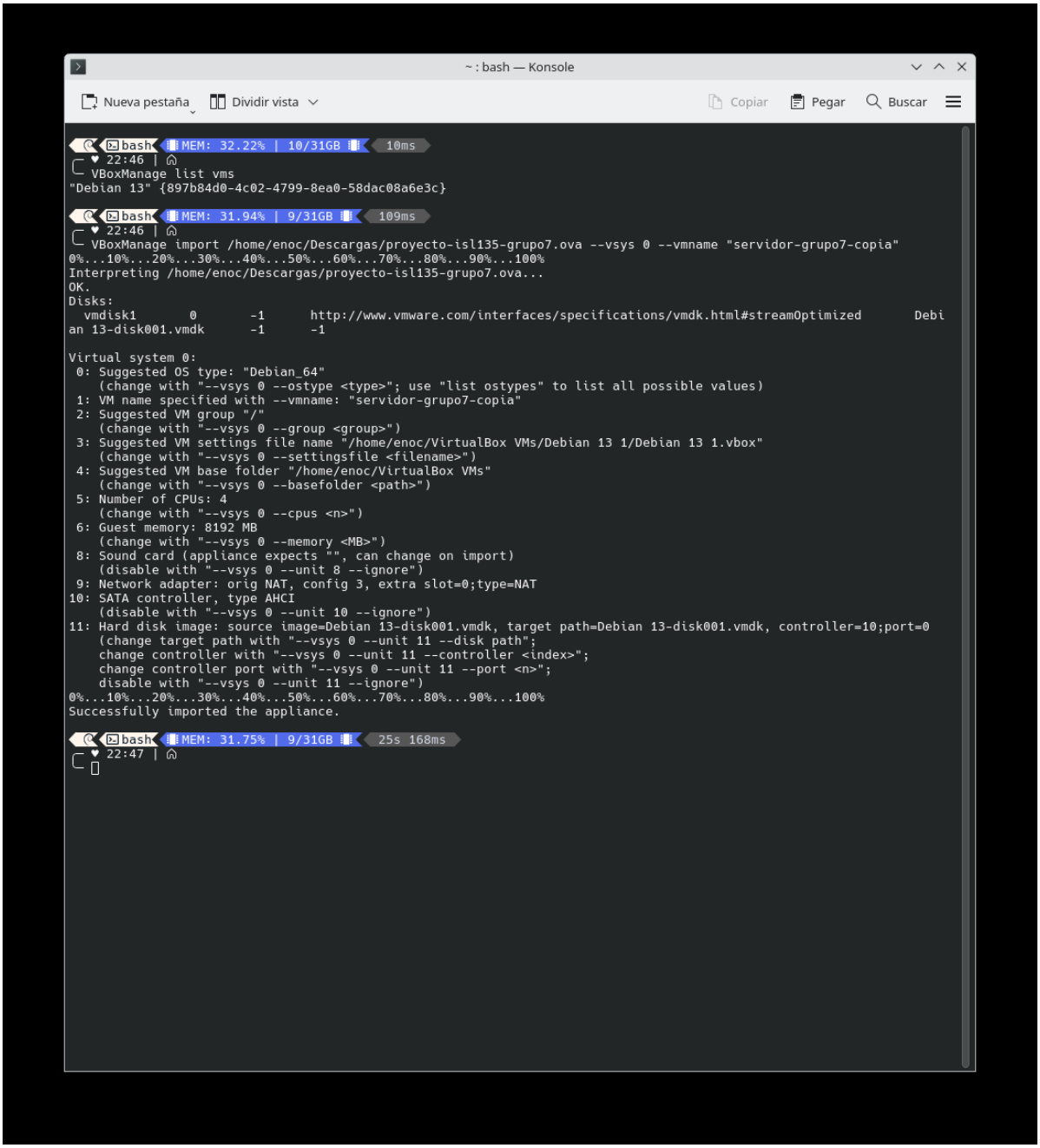


Figura 51. Importación de la OVA como copia independiente.

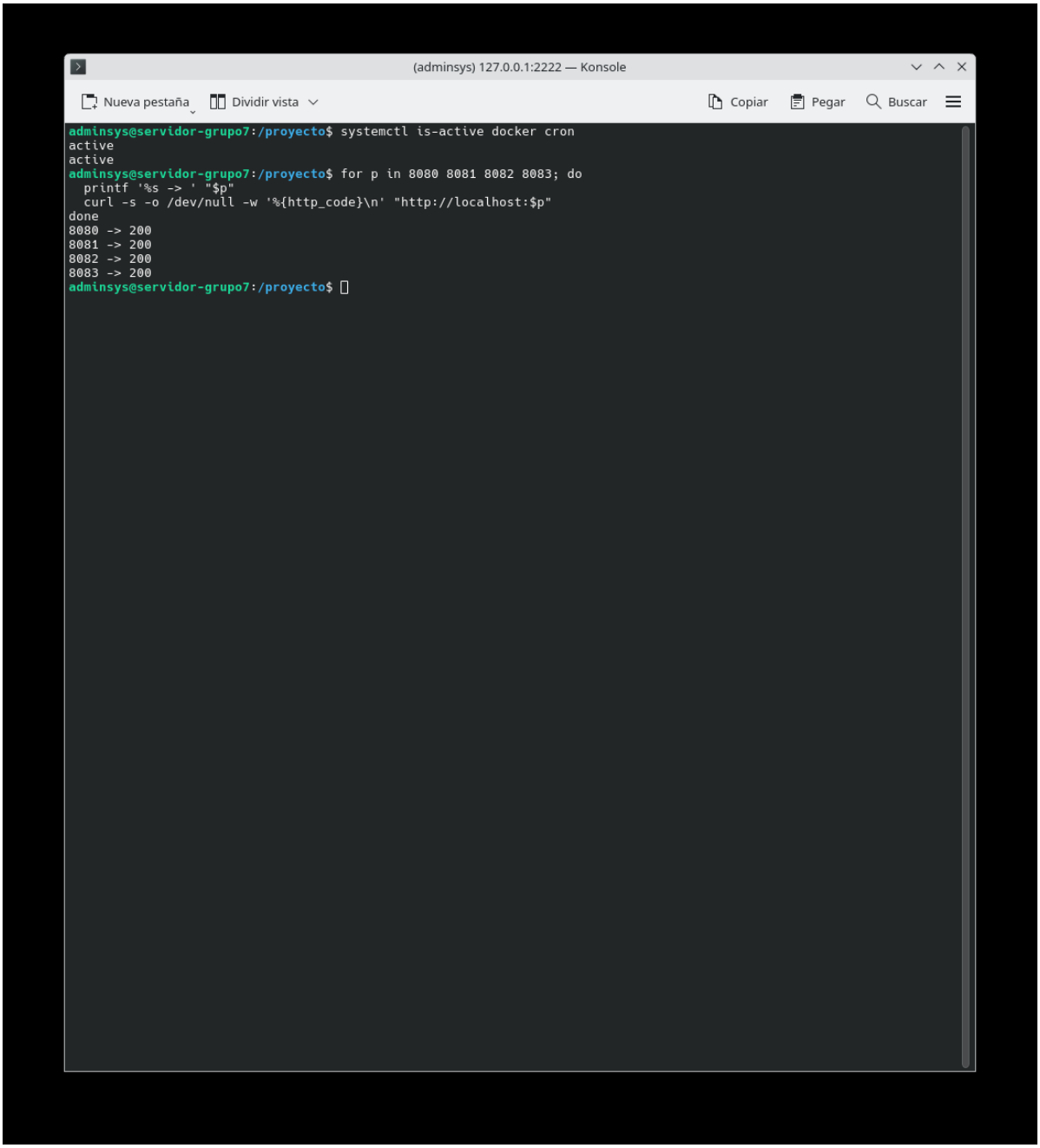


Figura 52. Cuatro puertos con HTTP 200 dentro de la copia.

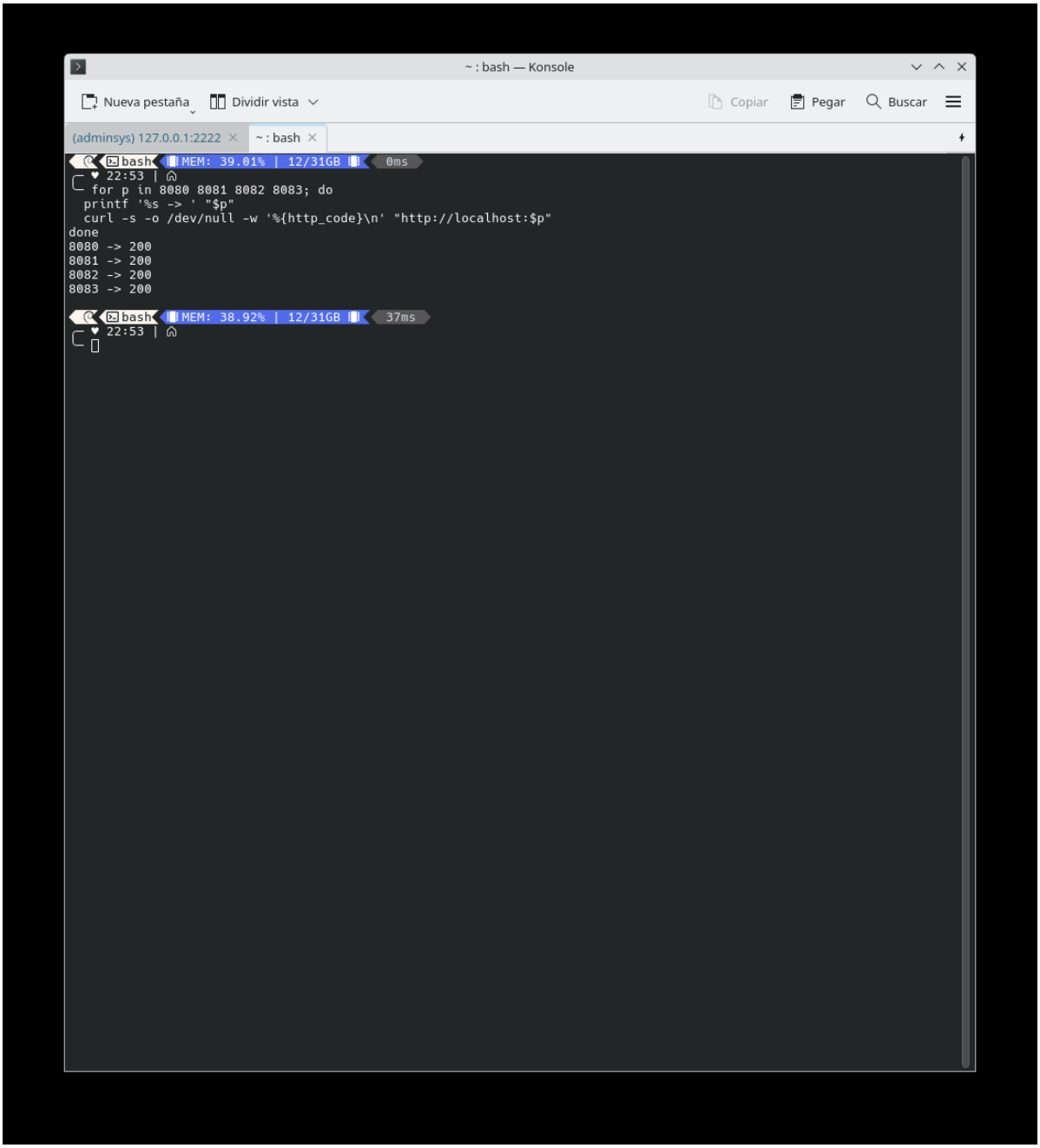


Figura 53. Cuatro puertos con HTTP 200 desde el host.

Conclusión de la fase

La OVA final se exportó, se verificó con SHA-256 y se importó como una copia independiente. Los cuatro servicios respondieron HTTP 200 tanto dentro de la copia como desde el host.

Resultados

Servicio	Puerto	Estado	Evidencia

Nginx básico con bind mount	8080	HTTP 200	capturas/fase-05-nginx-basico/
Imagen personalizada	8081	HTTP 200	capturas/fase-06-imagen-personalizada/
Red personalizada	8082	HTTP 200	capturas/fase-07-redes-volumenes/
Volumen persistente	8083	HTTP 200	capturas/fase-07-redes-volumenes/
Verificación integral	8080–8083	Aprobada	capturas/fase-08-pruebas-verificacion/

Los cuatro contenedores quedaron activos, cron se ejecuta cada 30 minutos, el repositorio está limpio y los servicios vuelven a arrancar después de reiniciar la máquina virtual.

Dificultades y soluciones

12. Repositorio Docker para Debian Trixie: la URL inicial devolvía 404. Se corrigió la suite a trixie y se validó la clave GPG antes de instalar.
13. Snapshot de la fase 5 perdido: VirtualBox perdió el registro histórico. Se creó el punto consolidado 05-nginx-recuperado y se continuó sin bloquear la fase 6.
14. Cambios pendientes en Git durante la verificación: el verificador detectó archivos sin commit. Se prepararon, se commitearon y se volvió a ejecutar la verificación hasta obtener código 0.

Conclusiones

- El servidor quedó funcionando con cuatro servicios web en Docker.
- La automatización con cron registra el estado del sistema cada 30 minutos.
- Git permitió el trabajo colaborativo y la trazabilidad de cada fase.
- Docker facilitó la reproducción del entorno y la separación de técnicas: bind mount, imagen propia, red personalizada y volumen persistente.
- La OVA se exportó, se verificó con SHA-256 y se importó como copia independiente, demostrando portabilidad.

Recomendaciones y trabajo futuro

- Usar docker compose para declarar los cuatro servicios en un solo archivo.
- Fijar versiones de imágenes en lugar de latest.
- Implementar HTTPS con Let's Encrypt.
- Usar un usuario no root dentro de los contenedores.
- Gestionar secretos con Docker secrets o un gestor externo.
- Agregar respaldos del volumen y monitoreo con Prometheus/Grafana.
- Automatizar la verificación con CI/CD.

Bibliografía

- Debian Project. *Debian GNU/Linux Documentation*. https://www.debian.org/doc/
- Docker Inc. *Docker Documentation*. https://docs.docker.com/
- Nginx. *Nginx Documentation*. https://nginx.org/en/docs/
- Git. *Pro Git*. https://git-scm.com/book/es/v2
- GitHub. *GitHub Docs*. https://docs.github.com/
- Universidad de El Salvador. *Guía del Proyecto Final ISL135*, Ciclo II 2026.

Anexos

Anexo A — Script de monitoreo

```
#!/usr/bin/env bash
#
# reporte_sistema.sh - Reporte básico del servidor
# Proyecto final ISL135 - Grupo 7
#
set -u

FECHA="$(date '+%Y-%m-%d %H:%M:%S')"
HOST="$(hostname)"
USUARIOS="$(who | wc -l)"

DISCO_TOTAL="$(df -h / | awk 'NR==2 {print $2}')"
DISCO_LIBRE="$(df -h / | awk 'NR==2 {print $4}')"
DISCO_USO="$(df -h / | awk 'NR==2 {print $5}')"

RAM_TOTAL="$(free -h | awk '/^Mem:/ {print $2}')"
RAM_DISPONIBLE="$(free -h | awk '/^Mem:/ {print $7}')"

if ! command -v docker >/dev/null 2>&1; then
    CONTENEDORES="Docker no instalado"
elif ! docker info >/dev/null 2>&1; then
    CONTENEDORES="No disponible (daemon detenido o sin permiso)"
else
    CONTENEDORES="$(docker ps -q | wc -l)"
fi

echo "================================================"
```

```
echo " REPORTE DEL SISTEMA - ${FECHA}"
echo "================================================"
printf " Hostname.............: %s\n" "${HOST}"
printf " Usuarios conectados.....: %s\n" "${USUARIOS}"
printf " Disco principal.........: %s libres de %s (%s usado)\n" "${DISCO_LIBRE}" "${DISCO_TOTAL}"
"${DISCO_USO}"
printf " RAM disponible..........: %s de %s\n" "${RAM_DISPONIBLE}" "${RAM_TOTAL}"
printf " Contenedores Docker.....: %s\n" "${CONTENEDORES}"
echo "================================================"
```

Anexo B — Dockerfile

```
FROM nginx:latest

LABEL maintainer="Grupo 7 - ISL135"
LABEL description="Servidor web personalizado del proyecto final"

# Copia el sitio dentro de la imagen
COPY web/ /usr/share/nginx/html/

# Documenta el puerto que expone Nginx
EXPOSE 80

# Mantiene Nginx en primer plano
CMD ["nginx", "-g", "daemon off;"]
```

Anexo C — Script de verificación integral

```
#!/usr/bin/env bash
#
# verificar_proyecto.sh - Verificación integral del proyecto ISL135
# Ejecutar como adminsys (con acceso a Docker) desde la VM:
#   /proyecto/scripts/verificar_proyecto.sh
#
# Devuelve 0 si todas las comprobaciones obligatorias pasan y 1 si hay fallos.
# No modifica el sistema ni los contenedores.
#
set -u

VERDE=$'\033[0;32m'
ROJO=$'\033[0;31m'
AMARILLO=$'\033[1;33m'
NC=$'\033[0m'

fallos=0
avisos=0

ok()    { printf '%s[OK]%s %s\n'    "$VERDE"    "$NC" "$*"; }
fallo() { printf '%s[FALLO]%s %s\n' "$ROJO"     "$NC" "$*"; fallos=$((fallos + 1)); }
aviso() { printf '%s[AVISO]%s %s\n' "$AMARILLO" "$NC" "$*"; avisos=$((avisos + 1)); }
info()  { printf '%s[INFO]%s %s\n'  "$AMARILLO" "$NC" "$*"; }

echo "================================================"
echo " VERIFICACIÓN INTEGRAL - PROYECTO ISL135"
echo " Fecha: $(date '+%Y-%m-%d %H:%M:%S')"
```

```
echo " Host:  $(hostname)"
echo " Usuario: $(id -un)"
echo "================================================"

if [[ "$(id -un)" != "adminsys" ]]; then
    aviso "Ejecuta como adminsys para comprobar Docker sin sudo; usuario actual: $(id -un)"
fi

# 1. Hostname
if [[ "$(hostname)" =~ ^servidor-grupo[0-9]+$ ]]; then
    ok "Hostname correcto: $(hostname)"
else
    fallo "Hostname inesperado: $(hostname) (se esperaba servidor-grupoN)"
fi

# 2. Usuarios y grupos
for u in adminsys tecnico visitante; do
    if id "$u" >/dev/null 2>&1; then
        ok "Existe el usuario $u"
    else
        fallo "Falta el usuario $u"
    fi
done

id -nG adminsys  2>/dev/null | grep -qw sudo    && ok "adminsys pertenece a sudo"    || fallo
"adminsys no pertenece a sudo"
id -nG tecnico   2>/dev/null | grep -qw soporte && ok "tecnico pertenece a soporte"  || fallo
"tecnico no pertenece a soporte"
id -nG visitante 2>/dev/null | grep -qw web     && ok "visitante pertenece a web"    || fallo
"visitante no pertenece a web"

for u in adminsys tecnico; do
    id -nG "$u" 2>/dev/null | grep -qw docker \
        && ok "$u pertenece al grupo docker" \
        || fallo "$u no pertenece al grupo docker"
done

# 3. Directorios, grupo propietario y setgid
for d in datos web scripts capturas; do
    if [[ -d "/proyecto/$d" ]]; then
        ok "Existe /proyecto/$d"
    else
        fallo "Falta /proyecto/$d"
    fi
done

if [[ -w /proyecto ]]; then
    ok "El usuario actual puede escribir en /proyecto"
else
    fallo "El usuario actual no puede escribir en /proyecto"
fi

grupo_datos="$(stat -c '%G' /proyecto/datos 2>/dev/null || true)"
grupo_web="$(stat -c '%G' /proyecto/web 2>/dev/null || true)"
perm_datos="$(stat -c '%A' /proyecto/datos 2>/dev/null || true)"
perm_web="$(stat -c '%A' /proyecto/web 2>/dev/null || true)"

[[ "$grupo_datos" == "soporte" ]] && ok "datos: grupo soporte" || fallo "datos: grupo incorrecto
($grupo_datos)"
```

```
[[ "$grupo_web" == "web" ]]       && ok "web: grupo web"       || fallo "web: grupo incorrecto
($grupo_web)"
[[ "$perm_datos" == drwxrws* ]]   && ok "datos tiene setgid ($perm_datos)" || fallo "datos sin
setgid ($perm_datos)"
[[ "$perm_web" == drwxrws* ]]     && ok "web tiene setgid ($perm_web)"     || fallo "web sin setgid
($perm_web)"

if [[ -f /proyecto/web/index.html ]]; then
    ok "Existe /proyecto/web/index.html"
else
    fallo "Falta /proyecto/web/index.html"
fi

if [[ -f /proyecto/Dockerfile ]]; then
    ok "Existe /proyecto/Dockerfile"
else
    fallo "Falta /proyecto/Dockerfile"
fi

# 4. Script de monitoreo
if [[ -x /proyecto/scripts/reporte_sistema.sh ]]; then
    ok "reporte_sistema.sh es ejecutable"

    salida_reporte="$(/proyecto/scripts/reporte_sistema.sh 2>&1)" || true
    for etiqueta in "REPORTE DEL SISTEMA" "Hostname" "Usuarios conectados" "Disco principal" "RAM
disponible" "Contenedores Docker"; do
        if grep -Fq "$etiqueta" <<<"$salida_reporte"; then
            ok "El reporte contiene: $etiqueta"
        else
            fallo "El reporte no contiene: $etiqueta"
        fi
    done
else
    fallo "reporte_sistema.sh no existe o no es ejecutable"
fi

# 5. Cron y log
if [[ -f /etc/cron.d/proyecto ]]; then
    ok "Existe /etc/cron.d/proyecto"
    if grep -Eq '^[^#]*\*/30[[:space:]].*reporte_sistema\.sh' /etc/cron.d/proyecto; then
        ok "Cron programado cada 30 minutos"
    else
        fallo "Cron no contiene una entrada */30 para reporte_sistema.sh"
    fi
else
    fallo "Falta /etc/cron.d/proyecto"
fi

systemctl is-active --quiet cron && ok "Servicio cron activo" || fallo "Servicio cron inactivo"
systemctl is-enabled --quiet cron && ok "Servicio cron habilitado" || fallo "Servicio cron no
habilitado"

log_cron="/var/log/proyecto/reporte_sistema.log"
if [[ -s "$log_cron" ]]; then
    ok "Log de cron con contenido ($(wc -l < "$log_cron") líneas)"
    if find "$log_cron" -mmin -35 -print -quit 2>/dev/null | grep -q .; then
        ok "Log actualizado en los últimos 35 minutos"
    else
        aviso "El log no se ha actualizado en los últimos 35 minutos"
```

```
    fi
else
    fallo "Log de cron vacío o ausente: $log_cron"
fi

# 6. Docker
if command -v docker >/dev/null 2>&1; then
    ok "Docker instalado: $(docker --version 2>/dev/null)"
    systemctl is-active --quiet docker && ok "Servicio docker activo" || fallo "Servicio docker
inactivo"
    systemctl is-enabled --quiet docker && ok "Servicio docker habilitado" || fallo "Servicio
docker no habilitado"

    if docker info >/dev/null 2>&1; then
        ok "El usuario actual puede usar Docker"

        if docker images --format '{{.Repository}}' | grep -Eq '^servidor-grupo[0-9]+$'; then
            ok "Existe una imagen servidor-grupoN"
        else
            fallo "No existe la imagen servidor-grupoN"
        fi
    else
        fallo "Sin acceso a Docker (revisa el grupo docker y vuelve a iniciar sesión)"
    fi
else
    fallo "Docker no está instalado"
fi

# 7. Contenedores, puertos, red y volumen
if command -v docker >/dev/null 2>&1 && docker info >/dev/null 2>&1; then
    for c in nginx-basico nginx-personalizado nginx-red nginx-volumen; do
        if docker ps --format '{{.Names}}' | grep -qx "$c"; then
            ok "Contenedor activo: $c"
        else
            fallo "Contenedor no activo: $c"
        fi
    done

    mount_basico="$(docker inspect -f '{{range .Mounts}}{{.Source}} ->
{{.Destination}}{{"\n"}}{{end}}' nginx-basico 2>/dev/null || true)"
    mount_red="$(docker inspect -f '{{range .Mounts}}{{.Source}} ->
{{.Destination}}{{"\n"}}{{end}}' nginx-red 2>/dev/null || true)"
    mount_volumen="$(docker inspect -f '{{range .Mounts}}{{.Name}} ->
{{.Destination}}{{"\n"}}{{end}}' nginx-volumen 2>/dev/null || true)"

    grep -Fq '/proyecto/web -> /usr/share/nginx/html' <<<"$mount_basico" \
        && ok "nginx-basico monta /proyecto/web" \
        || fallo "nginx-basico no monta /proyecto/web"
    grep -Fq '/proyecto/web -> /usr/share/nginx/html' <<<"$mount_red" \
        && ok "nginx-red monta /proyecto/web" \
        || aviso "nginx-red no monta /proyecto/web (no es obligatorio para la fase 7)"
    grep -Fq 'volumen-web -> /usr/share/nginx/html' <<<"$mount_volumen" \
        && ok "nginx-volumen usa volumen-web" \
        || fallo "nginx-volumen no usa volumen-web"

    for p in 8080 8081 8082 8083; do
        code="$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 "http://localhost:$p"
2>/dev/null || echo '000')"
        [[ "$code" == "200" ]] && ok "http://localhost:$p -> 200" || fallo "http://localhost:$p ->
```

```
$code"
    done

    docker network inspect red-proyecto >/dev/null 2>&1 \
        && ok "Red red-proyecto existe" \
        || fallo "Falta red red-proyecto"

    if docker inspect -f '{{json .NetworkSettings.Networks}}' nginx-red 2>/dev/null | grep -q 'red-
proyecto'; then
        ok "nginx-red está conectado a red-proyecto"
    else
        fallo "nginx-red no está conectado a red-proyecto"
    fi

    docker volume inspect volumen-web >/dev/null 2>&1 \
        && ok "Volumen volumen-web existe" \
        || fallo "Falta volumen volumen-web"

    if docker exec nginx-volumen test -s /usr/share/nginx/html/index.html >/dev/null 2>&1; then
        ok "El volumen contiene index.html"
    else
        fallo "El volumen no contiene index.html"
    fi
fi

# 8. Git
if [[ -d /proyecto/.git ]]; then
    ok "Repositorio Git en /proyecto"

    if git -C /proyecto remote get-url origin >/dev/null 2>&1; then
        ok "Remoto origin: $(git -C /proyecto remote get-url origin)"
    else
        fallo "Sin remoto origin"
    fi

    if [[ -f /proyecto/README.md ]]; then
        ok "Existe README.md"
    else
        fallo "Falta README.md"
    fi

    if [[ -f /proyecto/.gitignore ]]; then
        ok "Existe .gitignore"
    else
        fallo "Falta .gitignore"
    fi

    estado_git="$(git -C /proyecto status --porcelain 2>/dev/null || true)"
    if [[ -z "$estado_git" ]]; then
        ok "Git sin cambios pendientes"
    else
        fallo "Git tiene cambios sin commitear"
        printf '%s\n' "$estado_git"
    fi

    autores="$(git -C /proyecto log --format='%an' 2>/dev/null | sed '/^$/d' | sort -u | wc -l)"
    if (( autores >= 4 )); then
        ok "Commits de $autores autores distintos"
    else
```

```
        fallo "Solo $autores autores distintos; se esperan al menos 4"
    fi

    if git -C /proyecto ls-files | grep -Eq '\.(ova|vdi|iso)$'; then
        fallo "El repositorio contiene una imagen de VM (OVA/VDI/ISO)"
    else
        ok "El repositorio no contiene OVA/VDI/ISO"
    fi

    info "Últimos commits:"
    git -C /proyecto log --oneline -5 2>/dev/null || true
else
    fallo "No hay repositorio Git en /proyecto"
fi

echo "================================================"
if (( fallos == 0 )); then
    if (( avisos > 0 )); then
        printf '%sRESULTADO: todo lo obligatorio pasó, con %d aviso(s).%s\n' "$VERDE" "$avisos"
"$NC"
    else
        printf '%sRESULTADO: todas las verificaciones pasaron.%s\n' "$VERDE" "$NC"
    fi
    exit 0
else
    printf '%sRESULTADO: %d verificación(es) con fallo.%s\n' "$ROJO" "$fallos" "$NC"
    exit 1
fi
```

Anexo D — Repositorio

- GitHub: https://github.com/cecv9/proyecto-linux-grupo7
- README: README.md (../README.md)
- Evidencias: capturas/INDICE.md (../capturas/INDICE.md)
- Documentación oficial: docs/README.md (README.md)